1. INTRODUCTION

CHAPTER 1: INTRODUCTION

1.1 ABOUT THE INTERNSHIP COMPANY / ORGANIZATION

Global Quest Technologies: Building Coders. Creating Futures.

The internship was carried out at Global Quest Technologies, located in Helanka, NewTown, Bangalore. It is a well-established training and development organization dedicated to providing high-quality, industry-oriented technical education to students and fresh graduates. The organization focuses on enhancing employability by equipping learners with practical skills required in today's competitive IT industry.

Global Quest Technologies offers specialized training programs in various software domains, with a strong emphasis on Java Full Stack Development. The curriculum is carefully designed to cover both frontend and backend technologies, enabling students to gain a comprehensive understanding of web application development. The training includes technologies such as HTML, CSS, JavaScript, Bootstrap, Java, JDBC, Servlets, Hibernate, and Spring frameworks.

1.1.1 Company Background

Founded in 2015, Global Quest Technologies is a leading force in software development and IT training, headquartered in Bangalore—the tech capital of India. The organization specializes in delivering expert-led training, particularly in Core Java, helping learners thrive in today's dynamic software environment.

The institution combines academic knowledge with real-world application, aiming to build strong programming foundations and problem-solving capabilities among aspiring developers.

1.1.2 Training Methodology

The organization adopts a practical and project-based learning approach, where students are encouraged to work on real-time projects, case studies, and assignments. This approach helps learners apply theoretical knowledge in real-world scenarios and develop strong problem-solving skills. Experienced trainers and mentors with industry exposure provide continuous guidance. Regular assessments, coding practices, and mock interviews are conducted to prepare students for job placements.

1.1.3 Skill Development Focus

In addition to technical training, Global Quest Technologies emphasizes soft skills development, including:

- Communication skills
- Teamwork
- Professional ethics

This holistic training ensures that students are well-prepared for both technical and professional challenges in the IT industry.

1.1.4 Objective of the Organization

The primary objective of the organization is to bridge the gap between academic knowledge and industry requirements by delivering:

- Quality education
- Hands-on experience
- Career-oriented training programs

Through its structured learning methodology, the institute ensures students meet industry standards and secure promising career opportunities.

1.1.5 Mission

To deliver industry-aligned, hands-on Java training that builds strong programming foundations and prepares learners for real-world challenges in software development.

1.1.6 Vision

To be a benchmark in Java education by combining rigorous training, mentorship, and project-based learning that empowers a new generation of software professionals.

1.1.7 Pioneering Core Java Training Across Domains

Software Development:

The Core Java course forms the backbone for careers in backend development, application programming, and web technologies. With a strong emphasis on real-time projects and debugging practices, the program prepares students to be job-ready from day one.

Corporate Training:

The organization offers tailored Core Java programs to upskill teams and improve software quality. The training modules integrate best coding practices, design patterns, and real-time use cases.

Academic Bridging:

The organization collaborates with colleges and universities to bridge the gap between academic theory and industry needs. Through workshops and project-based learning, students are equipped with the coding skills that employers value.

1.2 OFFLINE INTERNSHIP DETAILS

The internship was conducted in offline mode at Global Quest Technologies, Bangalore, over a duration of 4 months. This in-person mode of training provided a highly interactive and engaging learning environment, allowing for direct communication with trainers and peers.

During the internship period, I actively participated in regular classroom sessions, practical lab exercises, and project development activities. The training primarily focused on Java Full Stack Development, covering both frontend and backend technologies. On the frontend side, I learned to design responsive and user-friendly interfaces using HTML, CSS, Bootstrap, and JavaScript. On the backend, I gained knowledge in Java programming, database connectivity using JDBC, and frameworks such as Hibernate and Spring. The offline mode of internship played a significant role in enhancing my learning experience. It allowed for real-time doubt clarification, immediate feedback from instructors, and better understanding of complex concepts through face-to-face interaction. Hands-on sessions and lab work enabled me to develop practical skills in coding, debugging, and application development.

In addition to technical training, the internship also focused on improving technical aptitude and problem-solving skills. Regular coding exercises, logical reasoning sessions, and aptitude tests were conducted to strengthen analytical thinking and prepare for technical interviews. These activities helped in improving speed, accuracy, and confidence in solving real-world problems.

Furthermore, the program emphasized the development of soft skills, which are essential for professional success. Sessions on communication skills, presentation techniques, teamwork, and time management were conducted. Group discussions, mock interviews, and collaborative projects helped in enhancing interpersonal skills and building confidence in a professional environment.

Working in a classroom environment encouraged collaborative learning, where I interacted with fellow trainees, shared ideas, and worked as part of a team on various assignments and projects. This experience greatly improved my communication skills, teamwork abilities, and overall personality.

Overall, the offline internship provided a comprehensive and immersive learning experience, combining technical knowledge, practical implementation, aptitude development, and soft skills training, thereby preparing me effectively for a career in the IT industry.

2. WORK DONE DURING INTERNSHIP

CHAPTER 2: WORK DONE DURING INTERNSHIP

2.1. ABOUT COURSE/DOMAIN

Java Full Stack Development

Java Full Stack Development refers to the process of building complete web applications by integrating both frontend and backend technologies. It provides a comprehensive understanding of application development, starting from designing user interfaces to handling server-side logic and database operations.

This domain enables developers to work on all layers of an application, making them versatile and industry-ready professionals capable of handling end-to-end development tasks.

2.1.1 Course Structure and Learning Approach

The course was structured in a progressive and systematic manner, ensuring a strong foundation before advancing to complex topics. The learning flow included:

- Core Java fundamentals
- Advanced Java concepts
- Frontend technologies
- Database management

This step-by-step approach helped in better understanding and practical implementation of full stack development concepts**.**

2.1.2 Core Java Fundamentals

The training began with Core Java, covering essential programming concepts such as:

- Data types and variables
- Operators and expressions
- Control statements (if, loops, switch)
- Arrays and methods

Object-Oriented Programming (OOP) concepts were emphasized, including:

- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Regular coding practice, assignments, and problem-solving exercises helped in writing efficient and optimized code.

2.1.3 Advanced Java Technologies

The Advanced Java module introduced enterprise-level technologies required for dynamic web applications:

- JDBC (Java Database Connectivity)
- Servlets
- Serialization

Additionally, important frameworks were covered:

- Hibernate (ORM framework)
- Spring Framework (Spring Core, Spring JDBC, Spring ORM)
- Spring Boot

These technologies provided knowledge of:

- Dependency Injection
- Loose Coupling
- Rapid Application Development

2.1.4 Database Management (MySQL)

The course included database training using MySQL, focusing on:

- Data Definition Language (DDL)
- Data Manipulation Language (DML)
- Joins and relationships
- Normalization
- Query optimization

This helped in integrating backend applications with databases efficiently.

2.1.5 Frontend Development Technologies

On the frontend side, the following technologies were used:

- HTML (structure of web pages)
- CSS (styling and layout)
- Bootstrap (responsive design)
- JavaScript (dynamic behavior and interactivity)

The training focused on building responsive, user-friendly, and interactive web interfaces using DOM manipulation and modern UI practices.

2.1.6 Practical Learning and Projects

The course emphasized **hands-on learning** through:

- Daily coding exercises

- Weekly assignments
- Mini-projects

This continuous practice helped in:

- Strengthening problem-solving skills
- Improving coding efficiency
- Gaining real-world development experience

2.1.7 Advantages of Java Full Stack Development

- Ability to work on both frontend and backend
- High demand in the IT industry
- Versatility in handling complete application development
- Better career opportunities as a full stack developer
- Strong foundation for advanced technologies

2.1.8 Limitations

- Requires learning multiple technologies
- Can be time-consuming to master all layers
- Requires continuous practice and updating skills

2.1.9 Industry Relevance

Java Full Stack Development is widely used in:

- Web application development
- Enterprise-level systems
- E-commerce platforms
- Banking and financial applications

It is highly in-demand due to its ability to handle complete application lifecycle development.

2.1.10 Career Opportunities

After completing this domain, various career roles can be pursued:

- Full Stack Developer
- Java Developer
- Backend Developer
- Web Application Developer
- Software Engineer

2.2 TECHNOLOGIES LEARNT DURING INTERNSHIP

During the internship at Global Quest Technologies, Bangalore, the focus was exclusively on mastering Core Java. The training was structured to provide a deep understanding of Java programming fundamentals, object-oriented design principles, and key APIs. The learning journey began with the basics and gradually progressed to more advanced features, equipping me with the skills necessary to develop robust, modular, and efficient Java applications.

Used Technologies

1. HTML
2. CSS
3. JavaScript
4. Core Java
5. Advanced Java
6. MySQL

• HTML (HyperText Markup Language)

HTML is the standard markup language used to create the structure of web pages. It defines the layout and content of a webpage using elements such as headings, paragraphs, links, images, and tables.

1. Create structured web pages using semantic tags like <header>, <section>, <article>, and <footer>

2. Design forms using input fields, labels, buttons, and validation attributes

3. Embed multimedia elements such as images and videos

4. Organize content using lists and tables

HTML acts as the backbone of any web application, providing the basic structure upon which styling and functionality are built.

Basic Structure of HTML

Every HTML document follows a standard structure:

<!DOCTYPE html>

```
<html>
<head>
  <title>My Page</title>
</head>
<body>
  <h1>Hello World</h1>
</body>
</html>
```

Important Tags

- <h1> to <h6> → Headings
- <p> → Paragraph
- <a> → Hyperlink
- <img> → Image
- , <ol>, <li> → Lists
- <table> → Tables

Forms in HTML

Forms are used to collect user input:

```
<form>
  Name: <input type="text" />
  Password: <input type="password" />
  <button>Submit</button>
</form>
```

Semantic HTML

Semantic tags improve readability and SEO:

1. <header>: Defines the top section of a page or section (usually contains title, logo, navigation).
2. <footer>: Defines the bottom section (usually contains copyright, links, contact info).
3. <section>: Represents a thematic grouping of content (like a chapter or topic).
4. <article>: Represents independent, self-contained content (like a blog post or news article).

CSS (Cascading Style Sheets)

CSS is used to style and design web pages. It controls the appearance of HTML elements, including colors, layouts, fonts, spacing, and responsiveness.

Key Concepts Learned

1. Applying styles using inline, internal, and external CSS
2. Using selectors, classes, and IDs to target elements
3. Implementing layouts using Flexbox and Grid
4. Creating responsive designs using media queries
5. Adding animations and transitions for better user experience

CSS enhances the visual appeal of web applications and ensures that applications are user-friendly and accessible across different devices.

Types of CSS

- Inline CSS
- Internal CSS
- External CSS

Ex:

```
p {
   color: blue;
   font-size: 16px;
}
```

Selectors

- Class selector → .className
- ID selector → #idName

Box Model

Every element consists of:

- Margin
- Border
- Padding
- Content

Flexbox

Used for layout alignment:

```
.container {
   display: flex;
   justify-content: center;
   align-items: center;
}
```

JavaScript

JavaScript is a programming language used to add interactivity and dynamic behavior to web pages.

During the internship, I worked on:

- DOM manipulation to dynamically update content
- Event handling such as clicks, form submissions, and user inputs
- Form validation to ensure correct data entry
- Basic functions, loops, and conditional statements
- Handling asynchronous operations using callbacks and promises

Variables and Data Types

Variables are used to store data values. JavaScript supports different types like string, number, boolean, etc.

Example:

```
let name = "Sai";   // String
```

```
let age = 22;       // Number

let isStudent = true; // Boolean
```

Functions

Functions are blocks of code that perform a specific task and can be reused.

Example:

```
function greet() {

   console.log("Hello");

}

greet();
```

DOM Manipulation

DOM allows JavaScript to access and modify HTML elements dynamically.

Example:

```
document.getElementById("demo").innerHTML = "Updated Content";
```

This changes the content of an HTML element.

Event Handling

Events are user actions like clicking a button or submitting a form. JavaScript responds using event handlers.

Example:

```
button.onclick = function() {

   alert("Clicked");

}
```

Form Validation

Form validation ensures that user inputs are correct before submission.

Example:

```
let name = "";

if(name === "") {

   alert("Name is required");

}
```

Conditional Statements

Used to make decisions in a program.

Example:

```
let age = 18;

if(age >= 18) {

   console.log("Eligible");

} else {

   console.log("Not Eligible");

}
```

Loops

Loops are used to repeat a block of code multiple times.

Example:

```
for(let i = 1; i <= 3; i++) {

   console.log(i);

}
```

Asynchronous Operations (Callbacks)

Used to handle tasks like API calls without stopping execution.

Example:

```
function greet(callback) {
```

```
console.log("Hello");

callback();}
```

SQL (Structured Query Language)

SQL is used to manage and manipulate relational databases. It allows developers to store, retrieve, and manage data efficiently.

I learned how to:

- Create and manage databases and tables
- Use queries like SELECT, INSERT, UPDATE, and DELETE
- Apply constraints such as PRIMARY KEY and FOREIGN KEY
- Perform joins to combine data from multiple tables
- Use aggregate functions like COUNT, SUM, and AVG

SQL is essential for backend development as it ensures efficient data storage and retrieval.

Java

Java is a powerful, object-oriented programming language used to build scalable and secure applications. During the internship, I developed a strong understanding of core Java concepts.

Java supports different types of variables based on their scope and usage:

- **Local Variables**: Declared inside methods and accessible only within that method.
- **Instance Variables**: Declared inside a class but outside methods; they belong to objects.
- **Static Variables**: Declared using the static keyword and shared among all instances of a class.

Example:

```
int localVar = 10;

class Student {

   int id;         // instance variable
```

```
    static String college = "GPREC";  // static variable
}
```

Data Types in Java

Java data types are broadly classified into two categories:

1. Primitive Data Types

Primitive data types are the basic building blocks of data in Java. They store simple values and are predefined by the language.

There are 8 primitive data types:

- int → stores integer values
- float → stores decimal values
- double → stores large decimal values
- char → stores a single character
- boolean → stores true or false
- byte, short, long → used for different ranges of integer values

These data types are efficient and consume less memory.

2. Non-Primitive Data Types

Non-primitive data types are more complex and are used to store references to objects.

Examples include:

- String → used to store text
- Arrays → used to store multiple values of the same type
- Classes and Objects → used to define custom data structures

Unlike primitive types, non-primitive types store references (memory addresses) rather than actual values.

Example:

```
String name = "Sai";
```

```
int[] numbers = {1, 2, 3, 4};

class Student {

   int id;

}
```

Type Casting in Java

Type casting is the process of converting one data type into another.

- **Implicit Casting (Widening)** → automatic conversion from smaller to larger type
- **Explicit Casting (Narrowing)** → manual conversion from larger to smaller type

Syntax:

```
int a = 10;

double b = a;   // implicit

double x = 10.5;

int y = (int) x;  // explicit
```

Object-Oriented Programming (OOP)

Object-Oriented Programming is a programming approach that uses objects and classes to design software. It helps in organizing code, improving reusability, and making programs easier to manage.

Why to Use OOP

- Helps in organizing code into classes and objects
- Provides code reusability through inheritance
- Ensures data security using encapsulation
- Makes code easy to maintain and modify
- Supports real-world modeling

Key Concepts of OOP

1. Class

A class is a blueprint used to create objects.

```
class Student {
    String name;
    int age;
}
```

2. Object

An object is an instance of a class.

```
Student s1 = new Student();
s1.name = "Sai";
s1.age = 22;
```

3. Encapsulation

Encapsulation means wrapping data and methods into a single unit and restricting access.

```
class Student {
    private String name;
    public void setName(String name) {
        this.name = name;
    }
    public String getName() {
        return name;
    }
}
```

4. Inheritance

Inheritance allows one class to reuse properties and methods of another class.

```
class Animal {
   void sound() {
      System.out.println("Animal makes sound");
   }
}
class Dog extends Animal {
   void bark() {
      System.out.println("Dog barks");
   }
}
```

5. Polymorphism

Polymorphism means one method behaving in different ways.

Types:

- Method Overloading (compile-time)
- Method Overriding (runtime)

```
class Math {
   int add(int a, int b) {
      return a + b;
   }
   int add(int a, int b, int c) {
      return a + b + c;
   }
```

```
}
```

6. Abstraction

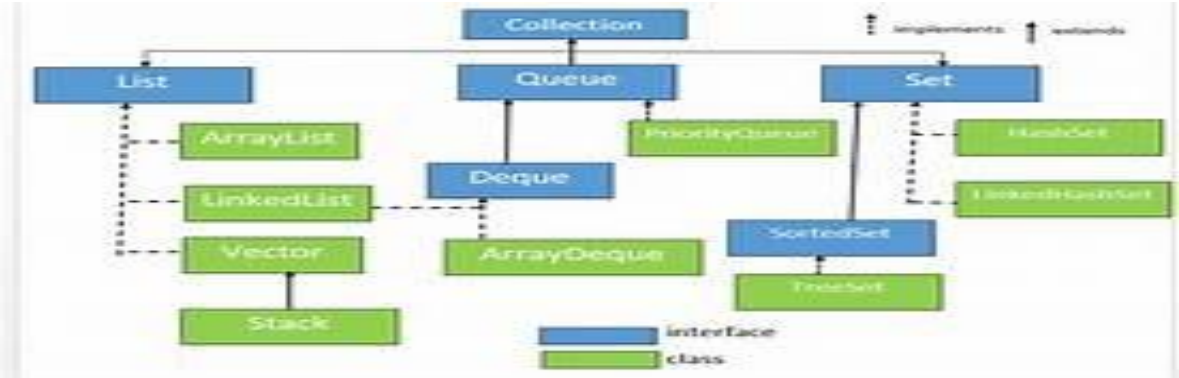
Abstraction means hiding implementation details and showing only essential features.

```
abstract class Vehicle {
    abstract void start();
}
class Car extends Vehicle {
    void start() {
        System.out.println("Car starts with key");
    }
}
```

Advantages of OOP

- Improves code reusability
- Makes code modular and easy to maintain
- Enhances security using encapsulation
- Helps in real-world modeling

Java Collections Framework



2.1 Java Collection Framework

Multithreading (Java)

Multithreading is a process of executing multiple threads simultaneously to perform tasks efficiently. A thread is the smallest unit of a program that runs independently.

It helps in:

- Improving performance
- Performing multiple tasks at the same time
- Better CPU utilization

1. Thread

A thread is a lightweight process.

```
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread is running");
   }
}
```

2. Creating Threads

Method 1: Extending Thread Class

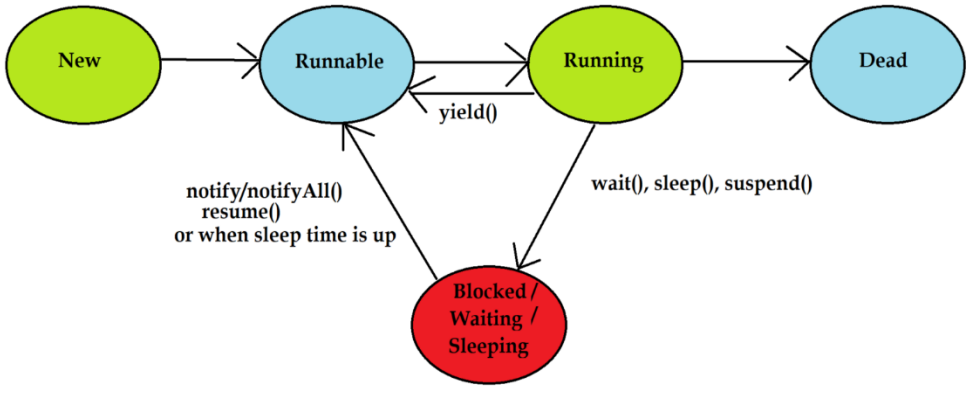
```
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread 1");
   }
}
MyThread t1 = new MyThread();
t1.start();
```

Method 2: Implementing Runnable Interface (Preferred)

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread 2");
    }
}
Thread t2 = new Thread(new MyRunnable());
t2.start();
```

3. Thread Lifecycle

- New
- Runnable
- Running
- Waiting / Blocked
- Terminated



Thread Lifecycle using Thread states

2.2 Thread LifeCyle

4. Thread Methods

- start() → starts thread
- run() → contains logic
- sleep() → pauses execution
- join() → waits for thread to finish

```
Thread.sleep(1000); // sleep for 1 second
```

5. Synchronization

Used to control access to shared resources and avoid conflicts.

```
synchronized void display() {

   System.out.println("Safe thread execution");

}
```

6. Advantages of Multithreading

- Faster execution
- Efficient resource usage
- Improves responsiveness

7. Real-Time Examples

- Downloading a file while watching a video
- Multiple users accessing a web application
- Backend APIs handling multiple requests

Exception Handling (Java)

Exception Handling is used to handle runtime errors so that the program does not crash and continues execution smoothly.

1. Exception

An exception is an error that occurs during execution.

```
int a = 10 / 0; // ArithmeticException
```

2. Types of Exceptions

- **Checked Exceptions** → Checked at compile time (IOException, SQLException)
- **Unchecked Exceptions** → Occur at runtime (ArithmeticException, NullPointerException)

3. try-catch Block

```
try {
   int a = 10 / 0;
} catch (Exception e) {
   System.out.println("Error occurred");
}
```

4. finally Block

```
try {
   int a = 10 / 0;
} catch (Exception e) {
   System.out.println("Handled");
} finally {
   System.out.println("Always executes");
}
```

5. throw Keyword

```
throw new ArithmeticException("Invalid operation");
```

6. throws Keyword

```
void check() throws Exception {

   // risky code

}
```

7. Custom Exception

User-defined exception created by extending Exception class.

Advantages of Exception Handling

- Maintains normal program flow
- Improves code readability
- Helps in debugging
- Separates error-handling code

Advanced Java

Advanced Java focuses on building dynamic web applications using technologies such as Servlets, JSP, and JDBC.
During the internship:

- Learned how Servlets handle client requests and responses
- Worked with JSP for dynamic web content
- Used JDBC for database connectivity
- Understood MVC architecture

1. JDBC (Java Database Connectivity)

JDBC is used to connect Java applications with a database.

Steps to Connect

a. Load Driver

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

b. Establish Connection

```
Connection con = DriverManager.getConnection(
   "jdbc:mysql://localhost:3306/dbname", "username", "password");
```

c. Create Statement

- Statement
- PreparedStatement
- CallableStatement

```
Statement stmt = con.createStatement();
```

d. Execute Query

- executeQuery() → SELECT
- executeUpdate() → INSERT, UPDATE, DELETE

```
ResultSet rs = stmt.executeQuery("SELECT * FROM student");
```

e. Process Result

```
while(rs.next()) {
   System.out.println(rs.getString("name"));
}
```

Hibernate

Hibernate simplifies database operations by providing an Object-Relational Mapping (ORM) framework. It eliminates the need to write complex SQL queries and helps developers interact with databases using Java objects. It is widely used in enterprise applications.

Why Hibernate?

• Reduces boilerplate code
• No need to write SQL queries
• Database independent (supports multiple databases)

• Improves performance using caching

• Easy integration with Spring

Key Features

• Object-Relational Mapping (ORM)

• Automatic Table Creation

• HQL (Hibernate Query Language)

• Caching Mechanism

DEPT OF CSE, GPREC, KNL Page|24

• Lazy Loading

Important Topics in Hibernate

1. Configuration

Used to configure Hibernate with database details.

Example:
hibernate.cfg.xml

```
<hibernate-configuration>

 <session-factory>

  <property name="connection.url">jdbc:mysql://localhost:3306/db</property>

  <property name="dialect">org.hibernate.dialect.MySQLDialect</property>

 </session-factory>

</hibernate-configuration>
```

2. Session Factory

Used to create Session objects. It is a heavyweight object created once.

3. Session

Used to perform database operations like insert, update, delete, and fetch data.

Example:

```
Session session = factory.openSession();

session.save(object);
```

4. Transaction

Ensures data consistency and integrity.

Example:

```
Transaction tx = session.beginTransaction();

tx.commit();
```

5. Entity Class

• Maps Java class to database table
• Uses annotations like @Entity, @Id

Example:

```
@Entity

class Student {

 @Id

 private int id;

}
```

6. Hibernate Annotations

Used instead of XML configuration.

Common annotations:
• @Entity
• @Table
• @Id
• @Column

7. HQL (Hibernate Query Language)

Used to perform database queries using object-oriented syntax.

Example:

from Student where id=1

8. CRUD Operations

Hibernate supports:
• save() → Insert
• update() → Update
• delete() → Delete
• get()/load() → Fetch

9. Caching

Improves performance by storing frequently accessed data.

Types:
• First-level cache (Session level)
• Second-level cache (SessionFactory level)

10. Lazy Loading

Loads data only when required, improving performance.

Spring Boot

Spring Boot simplifies backend development by providing built-in configurations and reducing boilerplate code. It is widely used for building REST APIs and microservices.

Why Spring Boot?

• No XML configuration
• Embedded server (Tomcat)
• Faster development
• Easy integration with databases
• Suitable for microservices

Key Features

• Auto Configuration
• Starter Dependencies
• Embedded Server

• Production-ready tools

Important Topics in Spring Boot

1. Spring Boot Starters

Predefined dependencies to simplify setup.

Examples:

• spring-boot-starter-web

• spring-boot-starter-data-jpa

• spring-boot-starter-security

2. Application Properties

Used to configure application settings.

Example:

server.port=8080

spring.datasource.url=jdbc:mysql://localhost:3306/db

3. REST Controllers

Used to handle HTTP requests and return responses.

Example:

```
@RestController
public class MyController {

@GetMapping("/hello")

public String hello() {

   return "Hello World";

}

}
```

4. Dependency Injection (DI)

Spring automatically injects required objects.

Achieved using @Autowired

Example:

```
@Autowired
private UserService service;
```

5. Spring Data JPA

Used for database operations without writing SQL.

Provides methods like:

- save()
- findAll()
- deleteById()

6. Entity and Repository

- Entity → Maps Java class to database table
- Repository → Interface to perform DB operations

Example:

```
@Entity
class User {

@Id

private int id;

}
```

7. Exception Handling

Used to handle errors globally.

Using @ControllerAdvice

8. Spring Security

Provides authentication and authorization.

Used for:

- Login systems
- Role-based access

9. Microservices

Spring Boot is widely used to build microservices architecture.

Each service runs independently.

10. Actuator

Provides production-ready features like:

• Health check

• Metrics

• Monitoring

Conclusion

Spring Boot simplifies Java development by providing ready-to-use tools, configurations, and features, making it ideal for building modern web applications and microservices.

In summary, Full Stack Java Development integrates frontend and backend technologies, tools, and methodologies to build powerful and responsive web applications. By utilizing modern frameworks like Spring Boot and React, adopting best practices, and collaborating effectively, developers can deliver high-quality software that meets user expectations and business goals efficiently.

Five Phases of Full Stack Java Development:

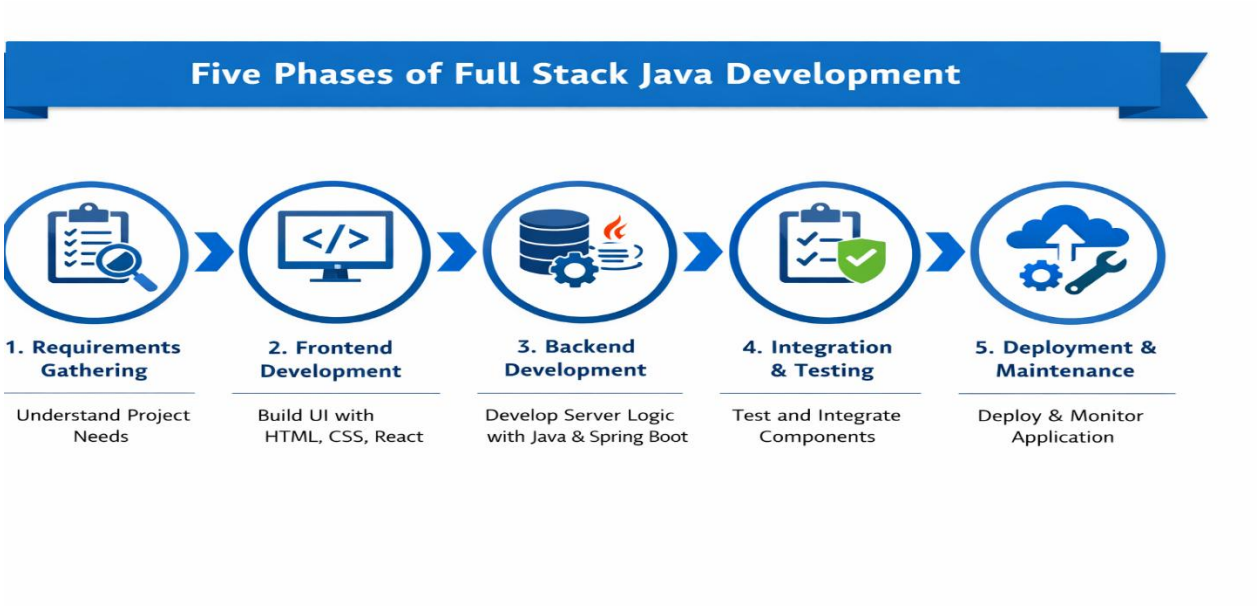


Fig 2.3: Phases of Full Stack Java Development

2.3 Assessments/Tasks assigned details

During the internship, regular assessments and tasks were assigned to strengthen both theoretical understanding and practical skills. In the Core Java phase, I consistently practiced coding by solving 10 programming challenges daily, which significantly improved my logical thinking and coding efficiency. Weekly assignments were also provided to reinforce concepts learned during the week.

In the Advanced Java phase, similar practice was followed with daily coding challenges and weekly assignments. In addition to this, I developed a project using Hibernate, which helped me understand real-time application development and database integration using ORM frameworks.

In the frontend domain, each topic was taught with clear explanations and practical examples. Weekly assignments were given that covered previously learned concepts, ensuring continuous revision and application of knowledge. Additionally, I completed two mini projects using HTML, CSS, Bootstrap, and JavaScript, which enhanced my ability to design and develop responsive web pages.

For the MySQL module, I practiced daily SQL queries and solved problems on the HackerRank platform, achieving certification up to the intermediate level. This improved my understanding of database operations, query optimization, and data handling.

I also maintained daily engagement on LinkedIn, where I shared learning updates, technical concepts, and progress during the internship. This helped me improve my professional communication, build consistency, and enhance my online presence in the tech community.

Overall, the internship included a combination of:

- Daily coding challenges
- Weekly assignments
- Mini projects and framework-based projects
- Continuous practice through online platforms
- Daily LinkedIn engagement

These activities greatly enhanced my coding skills, problem-solving ability, database knowledge, and full stack development understanding, making me more confident and industry-ready.

3. PROJECTS COMPLETED

Chapter 3: PROJECTS COMPLETED

3.1 Project 1: Console Based Quiz Application

1. Introduction

The Java Quiz Application is an interactive, console-based quiz program designed to simulate a level-based quiz game. The application allows a user to answer 10 multiple-choice questions sequentially, earning rewards for each correct answer. It incorporates lifelines similar to the popular quiz show "Who Wants to Be a Millionaire?", making the game more engaging and interactive. The main objective of this project is to demonstrate Java programming skills, including the use of loops, arrays, conditional statements, and object-oriented programming concepts.

2. Objectives

- Create a quiz program with 10 questions.
- Implement correct/wrong answer validation with color-coded messages.
- Implement two lifelines:
  - Audience Poll (shows percentage votes)
  - 50/50 (removes two wrong options)
- Restrict lifelines to one-time use per game, and disallow for the 10th question.
- Reward the user for each correct answer and display the total reward at the end.
- End the quiz immediately if the user provides a wrong answer.

3. Features

- User Details Input: The user is prompted for name and age.
- Questions Display: Questions are displayed sequentially with four options (a/b/c/d).
- Lifelines:
  - Audience Poll
  - 50/50
  - Lifeline option
- Audience Poll shows correct and wrong answer percentages in green/red.
- 50/50 displays only two options, highlighting the correct option in green and remaining wrong option in red.

- Color-coded Feedback:
  - Correct answer → Green message
  - Wrong answer → Red message
- Reward System: Each correct answer gives ₹1000, displayed alongside a personalized congratulations message.
- Quiz Termination: The quiz ends immediately on a wrong answer.

4. Implementation Details

Packages and Classes

1. Main.java
   - Handles the main loop of the quiz.
   - Displays questions, options, and lifeline prompts.
   - Validates user input and calls the reward system.
2. UserDetails.java
   - Prompts the user for name and age.
3. QuestionBank.java
   - Stores 10 questions, their options, and correct answers.
4. AnswerValidate.java
   - Checks if the user's answer matches the correct answer.
5. LifeLines.java
   - Implements the two lifelines:
     - Audience Poll: Displays percentage distribution.
     - 50/50: Removes two incorrect options
6. RewardSystem.java
   - Adds reward for each correct answer and displays personalized messages.

7. Colors.java
    - Stores ANSI color codes for console output (Red, Green, Reset).

5. Key Logic

1. Loop through 10 questions using a for loop.
2. Display questions and options for each question.
3. Show lifeline option e) if available (except for the 10th question).
4. Accept input from user:
    - a/b/c/d → answer
    - e → lifeline
5. For lifeline:
    - If Audience Poll → display options with percentages .
    - If 50/50 → displays only two options.
6. Validate answer:
    - Correct → show "Correct Answer!" and reward.
    - Wrong → show "Wrong Answer", terminate quiz.

6. Screenshots / Sample Outputs

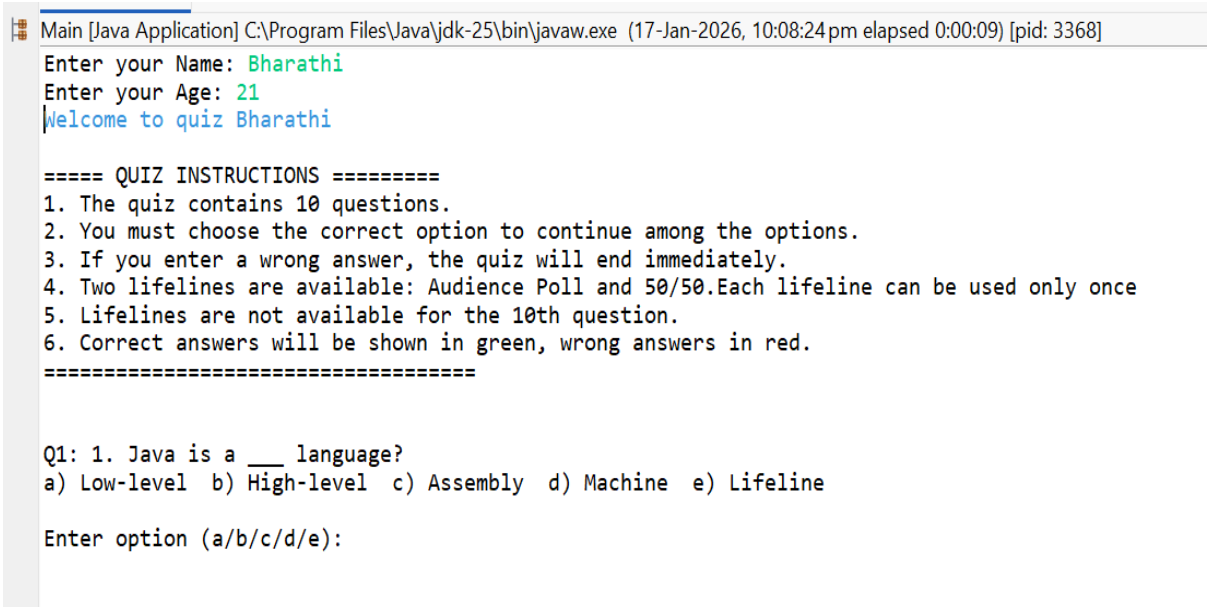


Fig 3.1 Welcome Message

```
Main [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe  (17-Jan-2026, 10:08:24 pm elapsed 0:00:44) [pid: 3368]
Enter your Name: Bharathi
Enter your Age: 21
Welcome to quiz Bharathi

===== QUIZ INSTRUCTIONS =========
1. The quiz contains 10 questions.
2. You must choose the correct option to continue among the options.
3. If you enter a wrong answer, the quiz will end immediately.
4. Two lifelines are available: Audience Poll and 50/50.Each lifeline can be used only once
5. Lifelines are not available for the 10th question.
6. Correct answers will be shown in green, wrong answers in red.
==================================


Q1: 1. Java is a ___ language?
a) Low-level  b) High-level  c) Assembly  d) Machine  e) Lifeline

Enter option (a/b/c/d/e): b
Correct Answer!
Congratulations Bharathi! You got ₹1000
Q2: 2. Which keyword is used to inherit a class?
a) super  b) this  c) extends  d) implements  e) Lifeline

Enter option (a/b/c/d/e):
```

Fig 3.2 Question Display

```
====================================


Q1: 1. Java is a ___ language?
a) Low-level  b) High-level  c) Assembly  d) Machine  e) Lifeline

Enter option (a/b/c/d/e): b
Correct Answer!
Congratulations Bharathi! You got ₹1000
Q2: 2. Which keyword is used to inherit a class?
a) super  b) this  c) extends  d) implements  e) Lifeline

Enter option (a/b/c/d/e): e

Choose Lifeline:
1) Audience Poll
2) 50/50
```

Fig 3.3 Lifelines

```
Congratulations Bharathi! You got ₹1000
Q2: 2. Which keyword is used to inherit a class?
a) super  b) this  c) extends  d) implements  e) Lifeline

Enter option (a/b/c/d/e): e

Choose Lifeline:
1) Audience Poll
2) 50/50
1

========= AUDIENCE POLL =================

a -> 10%  b -> 10%  c -> 70%   d -> 10%

=========================================


Q2: 2. Which keyword is used to inherit a class?
a) super  b) this  c) extends  d) implements
Enter option (a/b/c/d/e): c
Correct Answer!
Congratulations Bharathi! You got ₹2000
Q3: 3. Size of int data type in Java?
a) 2 bytes  b) 4 bytes  c) 8 bytes  d) Depends on OS  e) Lifeline

Enter option (a/b/c/d/e):
```

Fig 3.4 Audience Poll

```
Enter option (a/b/c/d/e): e

Choose Lifeline:
1) Audience Poll
2) 50/50
2
=========Fifty Fifty Activated=========

Q3: 3. Size of int data type in Java?
b) 4 bytes  c) 8 bytes
Enter option (a/b/c/d/e): b
Correct Answer!
Congratulations Bharathi! You got ₹3000
Q4: 4. Which operator is used for logical AND?
a) &  b) |  c) &&  d) ||  e) Lifeline

Enter option (a/b/c/d/e): c
Correct Answer!
Congratulations Bharathi! You got ₹4000
```

Fig 3.5 50/50 Lifeline

```
Q1: 1. Java is a ___ language?
a) Low-level  b) High-level  c) Assembly  d) Machine  e) Lifeline

Enter option (a/b/c/d/e): b
Correct Answer!
Congratulations Bharathi! You got ₹1000
Q2: 2. Which keyword is used to inherit a class?
a) super  b) this  c) extends  d) implements  e) Lifeline

Enter option (a/b/c/d/e): c
Correct Answer!
Congratulations Bharathi! You got ₹2000
Q3: 3. Size of int data type in Java?
a) 2 bytes  b) 4 bytes  c) 8 bytes  d) Depends on OS  e) Lifeline

Enter option (a/b/c/d/e): e
```

3.6 Correct Answer

```
Congratulations Bharathi! You got ₹3000
Q4: 4. Which operator is used for logical AND?
a) &  b) |  c) &&  d) ||  e) Lifeline

Enter option (a/b/c/d/e): c
Correct Answer!
Congratulations Bharathi! You got ₹4000
Q5: 5. Which company developed Java?
a) Google  b) Microsoft  c) Sun Microsystems  d) Apple  e) Lifeline

Enter option (a/b/c/d/e): b
Sorry your failed
Quiz Ended!

===== QUIZ COMPLETED =====
Congratulations Bharathi You won ₹4000
```

Fig 3.7 Wrong Answer

7. Challenges Faced

- Handling lifelines only once per game.
- Ensuring 50/50 removes only wrong answers and displays only two options.
- Implementing colored console output for different messages.
- Ending the quiz immediately on a wrong answer while keeping reward intact.

8. Tools & Environment Used

- Programming Language: Java
- IDE: Eclipse / IntelliJ IDEA
- Console: ANSI color codes for terminal output
- Data Structures: Arrays

9. Conclusion

- This project demonstrates Java basics and intermediate concepts, such as: Object-Oriented Programming, Conditional statements and loops, Arrays, User interaction via console, Simple UI using colors
- The lifelines feature makes the quiz interactive and simulates real-life quiz games.
- The program is robust, user-friendly, and visually appealing with color-coded outputs.

3.2 Project 2: Console Based Ecommerce Application

1. Introduction

The Java E-Commerce Application is a console-based shopping system designed to simulate an online shopping experience. The application allows users to browse different product categories, view sub-categories, select products, add items to a cart, and proceed to payment.

The main objective of this project is to demonstrate core Java concepts such as Object-Oriented Programming (OOP), inheritance, abstraction, polymorphism, arrays, and menu-driven programming. This application is user-friendly and structured into multiple modules like Home Decor, Fashion, and Stationery, each having its own sub-categories and product classes.

2. Objectives

- To design a menu-driven e-commerce application using Java
- To implement category and sub-category based product browsing
- To demonstrate OOP concepts such as inheritance, abstraction, and polymorphism
- To implement a cart system for adding and billing products
- To simulate a simple payment process
- To provide a structured and modular Java application

3. Features

- Main Menu with multiple categories
- Category Modules:
  - Home Decor
  - Fashion
  - Stationery
- Sub-category based product listing
- Product details display (brand, material, price, etc.)
- Add to Cart functionality
- Buy Now option
- Bill generation with total amount
- Payment simulation

- Exit and navigation options

4. Implementation Details

Packages and Classes

1. **Product.java**
   - Base class for all products
   - Stores common attributes like category, name, company, and price
   - Contains abstract display() method
2. **Cart.java**
   - Stores selected products using an array
   - Provides addProduct(), showBill(), and isEmpty() methods
   - Calculates and displays total bill amount
3. **Payment.java**
   - Simulates payment process
   - Displays payment success message
4. **HomeDecor.java**
   - Handles Home Decor category menu
   - Manages sub-categories: Wall Decor, Lighting, Furniture, Curtains
   - Displays products and handles user interaction
5. **WallDecor.java**
   - Represents Wall Decor products
   - Attributes: material, size
6. **Lighting.java**
   - Represents Lighting products
   - Attributes: type, watts
7. **Furniture.java**
   - Represents Furniture products
   - Attributes: material, warranty
8. **Curtains.java**
   - Represents Curtains products

- Attributes: fabric, color

9. **Fashion.java**

   - Handles Fashion category menu
   - Manages sub-categories like Men, Women, and Accessories

10. **Stationery.java**

    - Handles Stationery category menu
    - Manages sub-categories like Writing Tools, Paper Products, Office Supplies

5. Key Logic

1. Display main menu and category options
2. Navigate to selected category and sub-category
3. Display list of products using arrays
4. Allow user to select a product
5. Show complete product details using display() method
6. Provide options:
   - Add to Cart
   - Buy Now
7. Generate bill and calculate total amount
8. Proceed to payment or return to menu

6. Screenshots / Sample Outputs

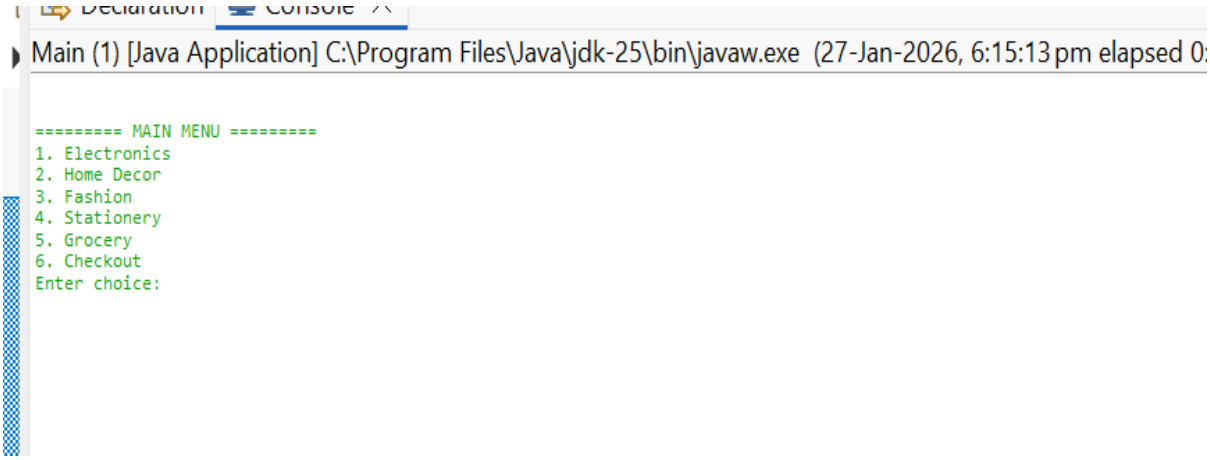


Fig 3.8 Main Menu Display

```
Main (1) [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe  (27-Jan-202


========== MAIN MENU ==========
1. Electronics
2. Home Decor
3. Fashion
4. Stationery
5. Grocery
6. Checkout
Enter choice:
1

--- ELECTRONICS SUB-CATEGORIES ---
1. Mobiles
2. Laptops
3. Headphones
4. Power Banks
0. Back to Main Menu
```

Fig 3.9 Category Selection

```
1

--- ELECTRONICS SUB-CATEGORIES ---
1. Mobiles
2. Laptops
3. Headphones
4. Power Banks
0. Back to Main Menu
2

--- LAPTOPS ---
1. Dell Inspiron
2. HP Pavilion
3. Lenovo IdeaPad
4. Asus VivoBook
5. MacBook Air
6. Acer Aspire
7. MSI Modern
8. Samsung Galaxy Book
9. Honor MagicBook
10. LG Gram
Select product (0-back): 2
```

Fig 3.10 Sub-Category Product List

```
--- LAPTOPS ---
1. Dell Inspiron
2. HP Pavilion
3. Lenovo IdeaPad
4. Asus VivoBook
5. MacBook Air
6. Acer Aspire
7. MSI Modern
8. Samsung Galaxy Book
9. Honor MagicBook
10. LG Gram
Select product (0-back): 2

Category : Electronics
Company  : HP
Model    : HP Pavilion
RAM      : 16 GB
Storage  : 512 GB
Price    : Rs.68999.0

1. Add to Cart
2. Buy Now
```

Fig 3.11 Product Details Display

```
10. LG Gram
Select product (0-back): 2

Category : Electronics
Company  : HP
Model    : HP Pavilion
RAM      : 16 GB
Storage  : 512 GB
Price    : Rs.68999.0

1. Add to Cart
2. Buy Now
1
✔ Product added to cart


--- ELECTRONICS SUB-CATEGORIES ---
1. Mobiles
2. Laptops
3. Headphones
4. Power Banks
0. Back to Main Menu
```

Fig 3.12 Add to Cart Confirmation

```
========== BILL RECEIPT ==========
Electronics | Samsung S23 | Rs.74999.0
Fashion | Jeans | Rs.2999.0
Home Decor | Printed Curtains | Rs.2199.0
Grocery | Mango | Rs.150.0
---------------------------------
TOTAL AMOUNT : Rs.80347.0
=================================


--- PAYMENT OPTIONS ---
1. Cash
2. UPI
3. Net Banking
```

Fig 3.13 Bill Receipt

```
=================================



--- PAYMENT OPTIONS ---
1. Cash
2. UPI
3. Net Banking
1
✔ Payment Successful
```

Fig 3.14 Payment Successful Message

7. Challenges Faced

- Designing a modular structure without using collections
- Managing navigation between menus
- Avoiding code duplication across categories
- Handling user input validation
- Maintaining clean and readable console output

8. Tools & Environment Used

Programming Language: Java

IDE: Eclipse / IntelliJ IDEA

Platform: Console-based application

Data Structures Used: Arrays

9. Conclusion

- This project successfully demonstrates the use of core Java concepts in a real-world application
- The modular structure makes the program easy to understand and extend
- The project enhances understanding of OOP principles such as inheritance, abstraction, and polymorphism
- The application provides a strong foundation for building advanced e-commerce systems in Java

3.3 Project 3: Student Management System Using Hibernate

1. Introduction

The Student Management System is a Hibernate-based Java application designed to manage student records efficiently with database integration. The main purpose of this project is to perform CRUD (Create, Read, Update, Delete) operations on student data using Object-Relational Mapping (ORM) technology.

This application demonstrates how Java objects can be mapped with database tables using Hibernate framework, eliminating the complexity of direct SQL queries. It provides an efficient and structured way to handle student information such as ID, name, gender, and other details.

2. Objectives

- To develop a Java-based application using Hibernate framework
- To perform CRUD operations on student records
- To demonstrate Object-Relational Mapping (ORM) concepts
- To integrate Java application with database efficiently
- To reduce complexity of JDBC using Hibernate
- To improve understanding of backend development

3. Features

- Add new student details
- Update existing student information
- Delete student records
- View student details
- Database connectivity using Hibernate
- Efficient ORM-based data management

- Simple and user-friendly backend structure

4. Implementation Details

Technologies Used

- Java (Core & Advanced Java)
- Hibernate Framework
- MySQL Database
- Eclipse

Main Components

1. **Student.java**
    - Entity class representing student table
    - Contains fields like id, name, gender, course, amount
2. **hibernate.cfg.xml**
    - Configuration file for database connection
    - Contains DB URL, username, password, dialect
3. **StudentDAO.java**
    - Contains database operations (CRUD methods)
    - Uses Hibernate Session for transactions
4. **Main.java**
    - Entry point of the application
    - Provides menu-driven operations

Key Logic

1. Configure Hibernate using configuration file
2. Create session factory and session objects
3. Perform CRUD operations using session methods
4. Map Java class with database table using annotations/XML
5. Commit transactions to save changes in database

5. Sample Outputs

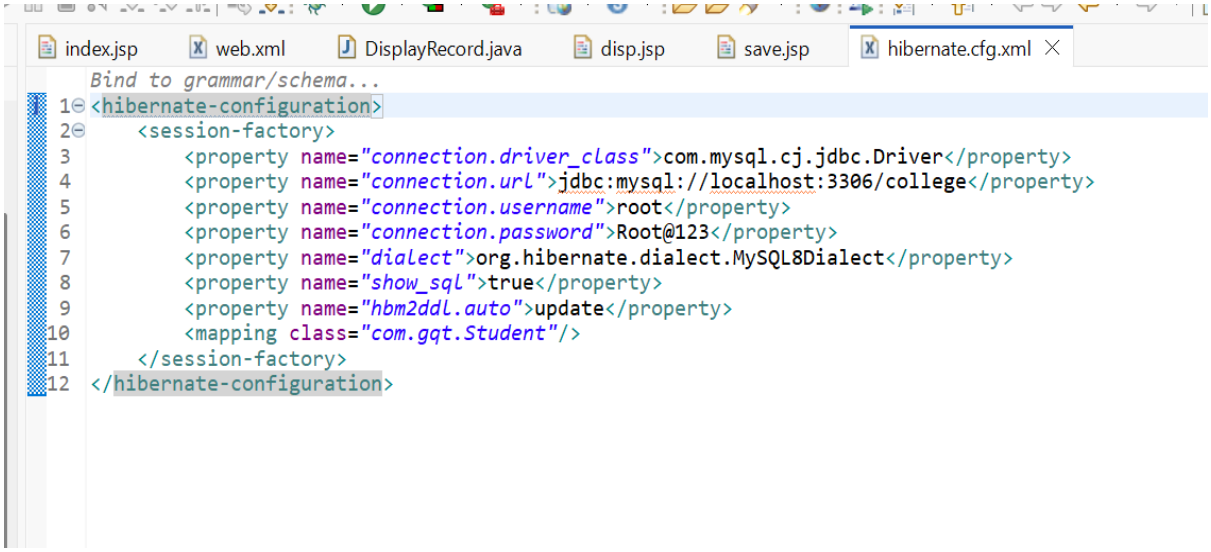


Fig 3.15 Hibernate.cfg.xml

```
mysql> use college;
Database changed
mysql> select * from student;
+-----+---------+--------+-----------+--------------------+--------+
| sid | name    | gender | city      | course             | amount |
+-----+---------+--------+-----------+--------------------+--------+
| 101 | valli   | Female | Bangalore | Java Full Stack    |  35000 |
| 103 | Amrutha | Female |           | Python Full Stack  |  32000 |
| 104 | sandhya | Female | Hyderabad | Java Full Stack    |  35000 |
| 105 | lalli   | Male   | Hyderabad | MySQL              |  12000 |
| 107 | Mahesh  | Male   | Hyderabad | Data Science       |  48000 |
| 108 | Mahesh  | Male   | Bangalore | Data Science       |  48000 |
| 109 | Mahesh  | Male   | Hyderabad | Data Analytics     |  23000 |
| 110 | Mahesh  | Male   | Hyderabad | Data Science       |  48000 |
| 112 | Mahesh  | Male   | Bangalore | Data Science       |  48000 |
| 113 | Mahesh  | Female | Hyderabad | Data Science       |  48000 |
+-----+---------+--------+-----------+--------------------+--------+
10 rows in set (0.01 sec)

mysql>
```

Fig 3.16 Database

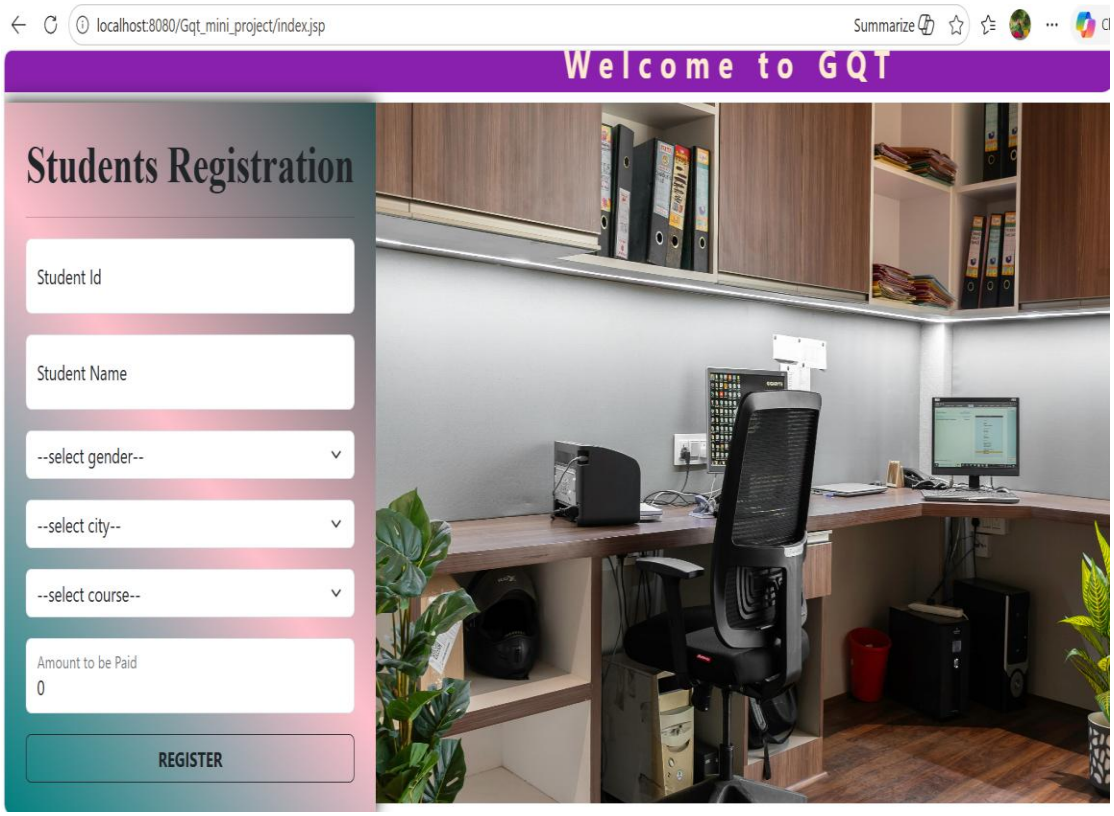


Fig 3.17 Insert Student Record

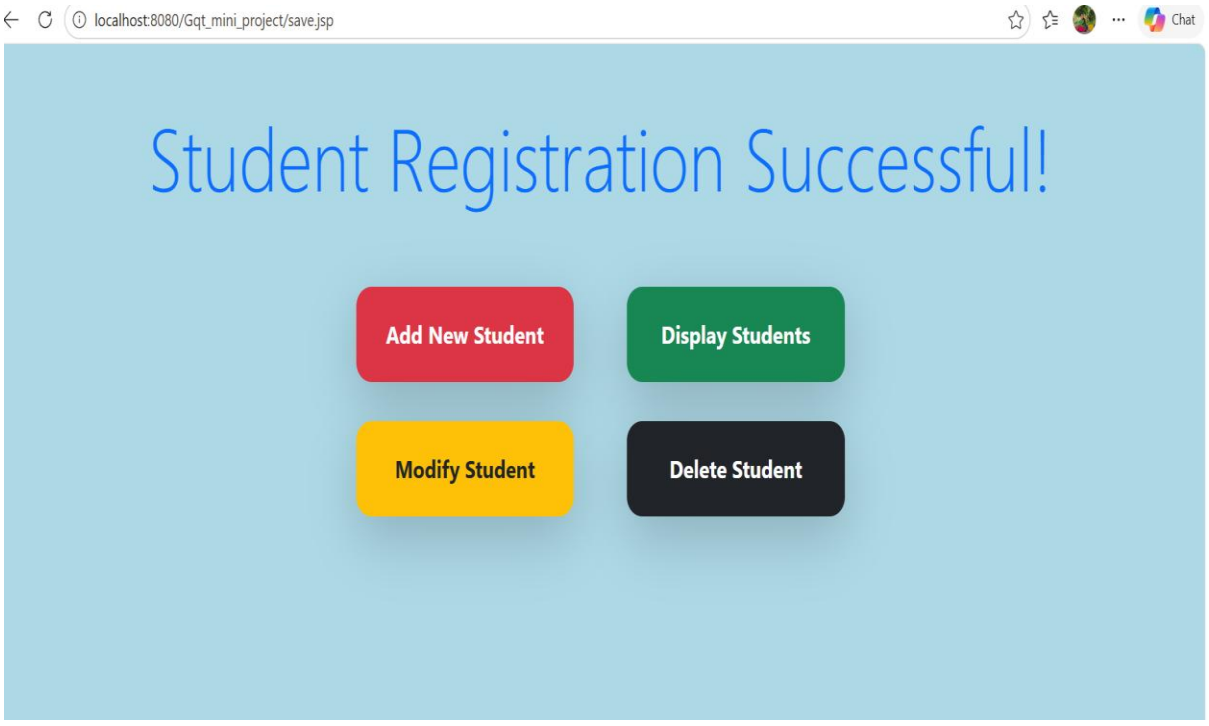


Fig 3.18 Main Page

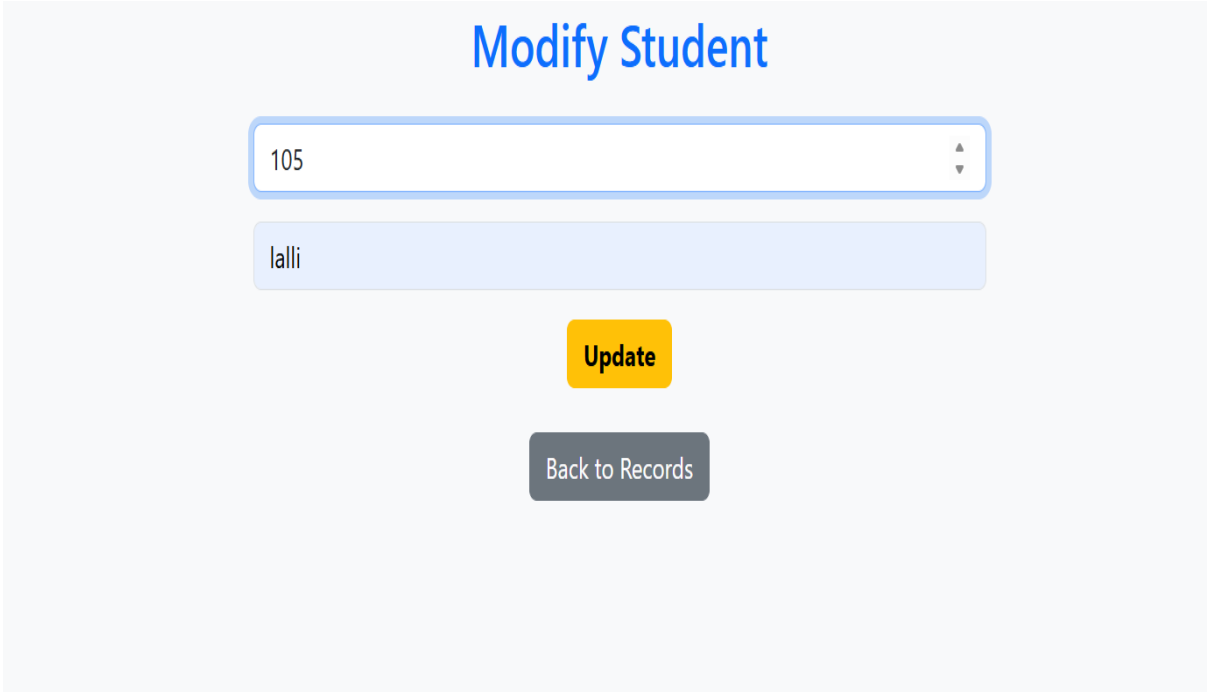
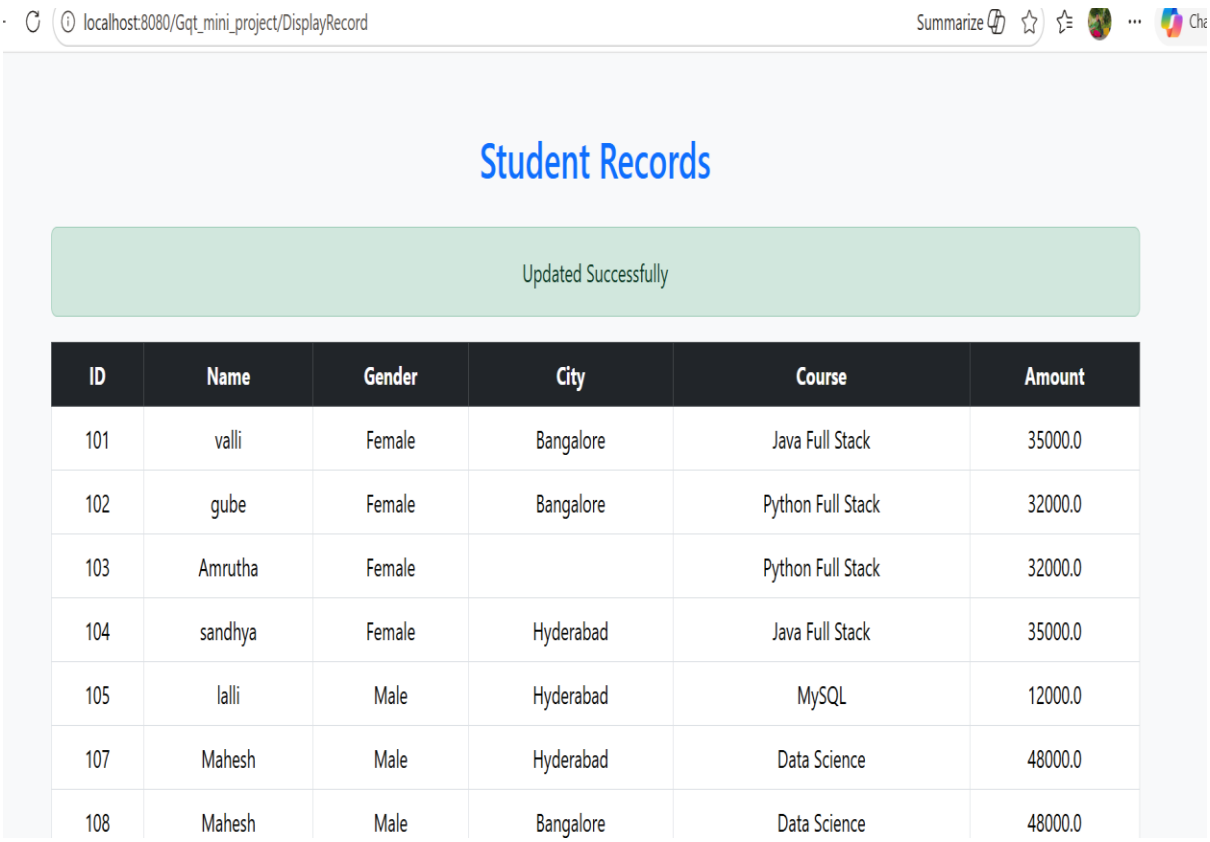


Fig 3.19A Update Student Details



ID	Name	Gender	City	Course	Amount
101	valli	Female	Bangalore	Java Full Stack	35000.0
102	gube	Female	Bangalore	Python Full Stack	32000.0
103	Amrutha	Female		Python Full Stack	32000.0
104	sandhya	Female	Hyderabad	Java Full Stack	35000.0
105	lalli	Male	Hyderabad	MySQL	12000.0
107	Mahesh	Male	Hyderabad	Data Science	48000.0
108	Mahesh	Male	Bangalore	Data Science	48000.0

Fig 3.19B Update Student Details

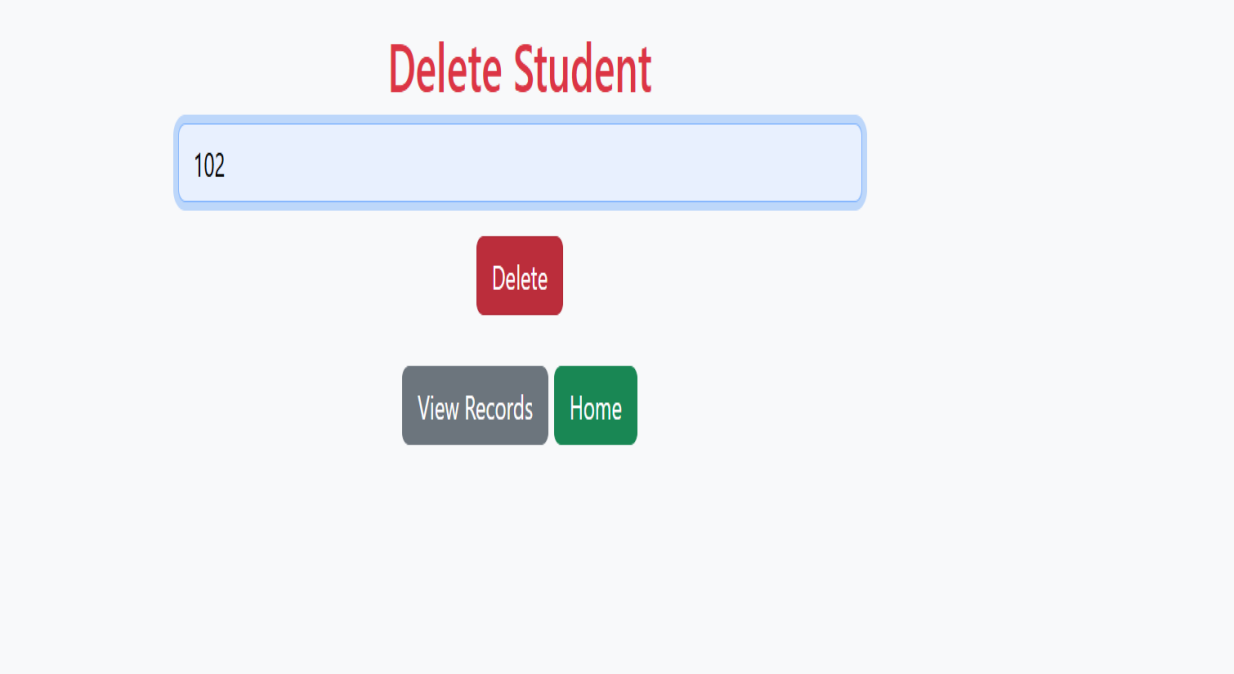
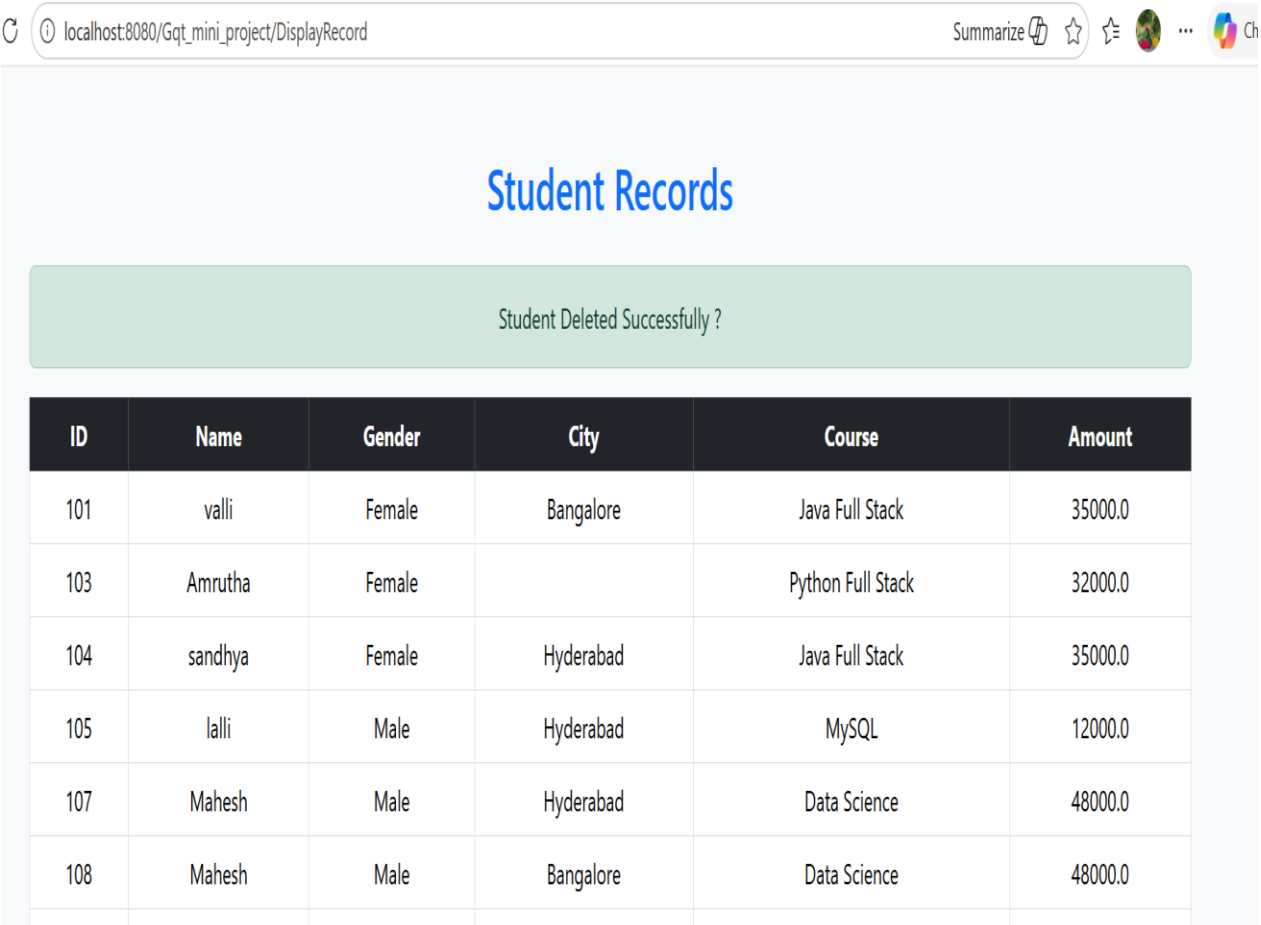


Fig 3.20A Delete Student Record



ID	Name	Gender	City	Course	Amount
101	valli	Female	Bangalore	Java Full Stack	35000.0
103	Amrutha	Female		Python Full Stack	32000.0
104	sandhya	Female	Hyderabad	Java Full Stack	35000.0
105	lalli	Male	Hyderabad	MySQL	12000.0
107	Mahesh	Male	Hyderabad	Data Science	48000.0
108	Mahesh	Male	Bangalore	Data Science	48000.0

Fig 3.20B Delete Student Record

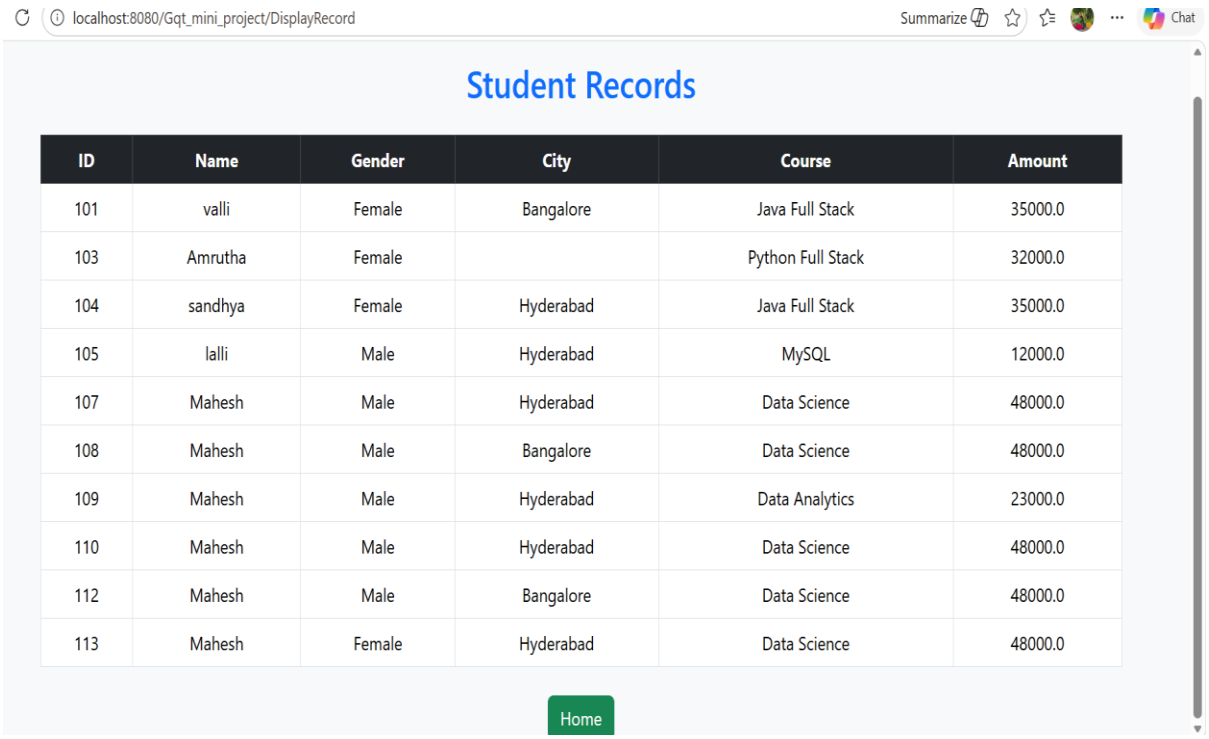


Fig 3.21 Understanding Hibernate configuration setup

6. Challenges Faced

- Understanding Hibernate configuration setup
- Mapping Java classes with database tables
- Managing sessions and transactions properly
- Debugging ORM-related errors
- Handling database connectivity issues

7. Tools & Environment Used

- Programming Language: Java
- Framework: Hibernate
- Database: MySQL
- IDE: Eclipse / IntelliJ IDEA
- Architecture: Console-based / Layered architecture

8. Learning Outcomes

- earned core Java and OOP concepts.
- Understood how to use the Hibernate Framework for database operations.
- Gained knowledge of CRUD operations using MySQL.
- Learned mapping between Java classes and database tables.
- Improved backend development and problem-solving skills.

9. Conclusion

This project helped me gain practical knowledge of Hibernate framework and ORM concepts. It improved my understanding of backend development and database integration.

The project also strengthened my skills in Java, database handling, and real-time application development. Overall, it provides a strong foundation for developing advanced enterprise-level applications using Spring and Hibernate frameworks.

3.4 Project 4: Website Development

3.4.1 A basic website using HTML and CSS

1. Introduction

This project is a front-end web application developed using HTML and CSS. It is designed as a structured web page that demonstrates basic website layout design and styling concepts. The main objective of this project is to build a simple multi-section website that represents an educational or training institute interface.

The application simulates a real-world website layout for Global Quest Technologies, including navigation menus, content sections, and informational blocks. It helps in understanding how web pages are structured and styled using frontend technologies.

2. Objectives

- To design a structured web page using HTML and CSS
- To implement navigation menus and multiple content sections
- To understand webpage layout design and styling concepts
- To practice responsive and organized UI development
- To improve frontend development skills
- To simulate a real institute website interface

3. Features

- Header section with institute name display
- Navigation menu with multiple links (Home, About Us, Courses, Internships, Careers, Contact Us)
- Content section introducing frontend technologies
- Multiple article sections for topics like HTML, CSS, Bootstrap, JavaScript
- Structured layout using div, section, and article tags
- External CSS styling using separate stylesheet
- Simple and clean UI design

4. Implementation Details

4.1 Technologies Used

- HTML (Hypertext Markup Language)
- CSS (Cascading Style Sheets)
- Font Awesome – Used to include attractive icons such as navigation icons, social media icons, and UI elements to enhance the visual appearance of the website.
- Animate.css – Used to apply animations like fade-in, slide, bounce, and zoom effects to webpage elements for better user experience and interactivity.

4.2 Project Structure

1. **index.html**
    - Main homepage of the project
    - Contains navigation bar, header, and content sections
2. **index-style.css / style.css**
    - Used for styling the webpage
    - Controls layout, colors, spacing, and design
3. **contact.html**
    - Separate contact page linked from navigation menu
4. **images folder**
    - Used for storing image assets

4.3 Key Elements Used

- Header with marquee text for institute name
- Navigation bar with anchor links

- Container-based layout structure
- Section and article tags for content organization
- Multiple topic blocks (HTML, CSS, Bootstrap, JavaScript, React-js)
- External CSS file for styling separation
- Use of Font Awesome icons for enhancing UI elements like menu items, buttons, and contact sections
- Use of CSS animations (Animate.css) to create dynamic effects such as fade-in, slide-up, and hover animations
- Responsive and visually appealing design using icons and animation effects

4.4 Enhancements Using External Libraries

- To improve the visual appearance and interactivity of the website, external libraries such as Font Awesome and Animate.css were integrated into the project.
- **Font Awesome:** Font Awesome is an icon toolkit used to add scalable vector icons to web pages. In this project, icons were used in navigation menus, buttons, and contact sections to make the interface more attractive and user-friendly.
- **Animate.css:** Animate.css is a CSS animation library that provides pre-built animations. It was used to add effects such as fade-in, bounce, and slide transitions to various webpage elements, improving user engagement and making the website more dynamic.

5. Output / Screenshots

Fig 3.22 Home Page with header and navigation menu

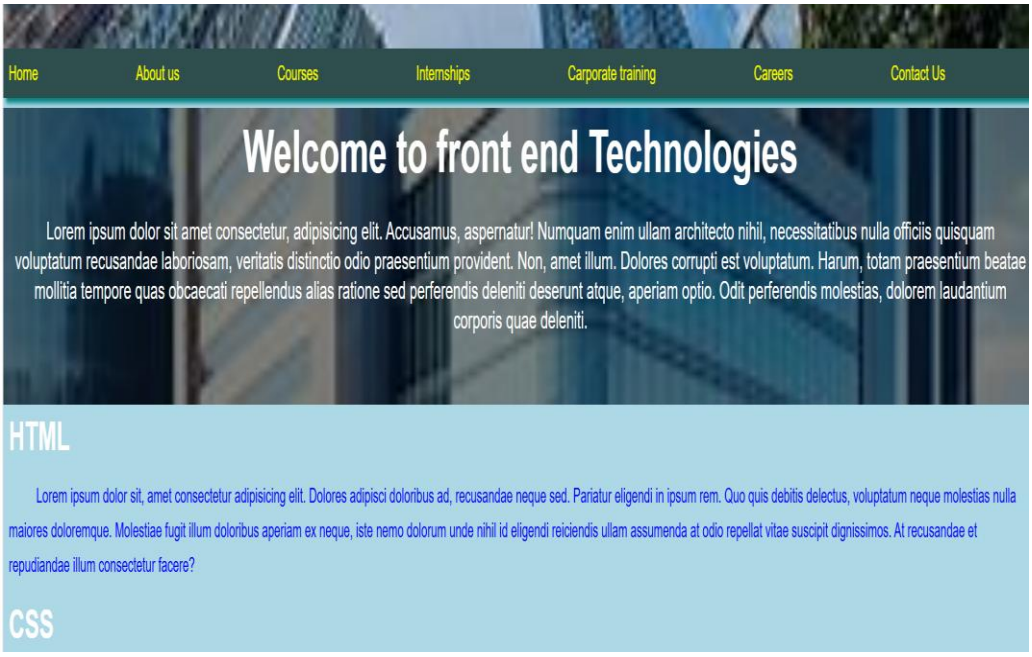


Fig 3.23 Content section displaying frontend introduction

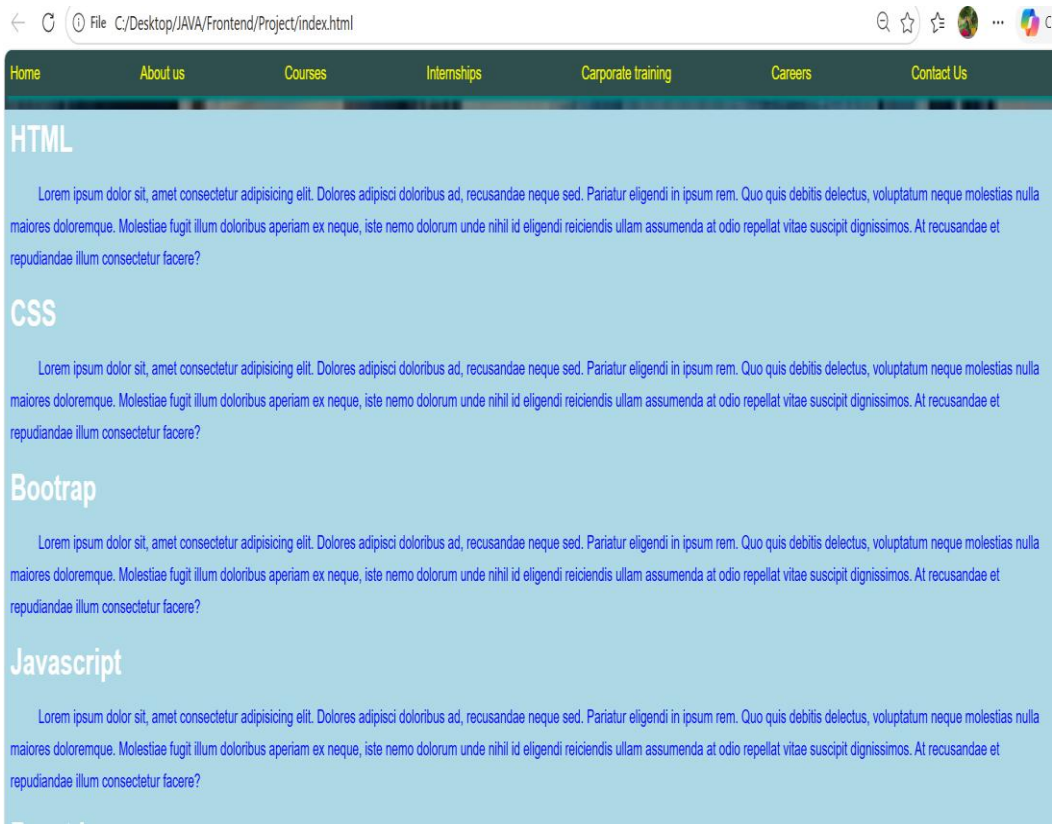


Fig 3.24 Multiple topic cards (HTML, CSS, Bootstrap, JavaScript, React-js)

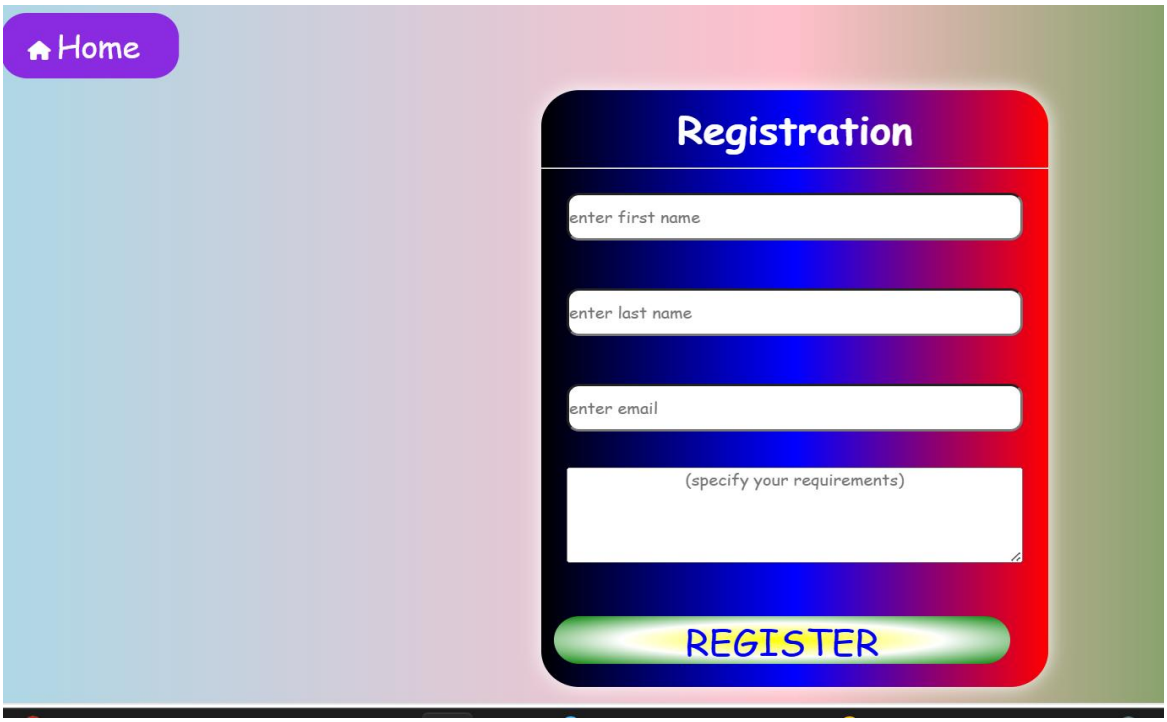


Fig 3.25 Contact page layout

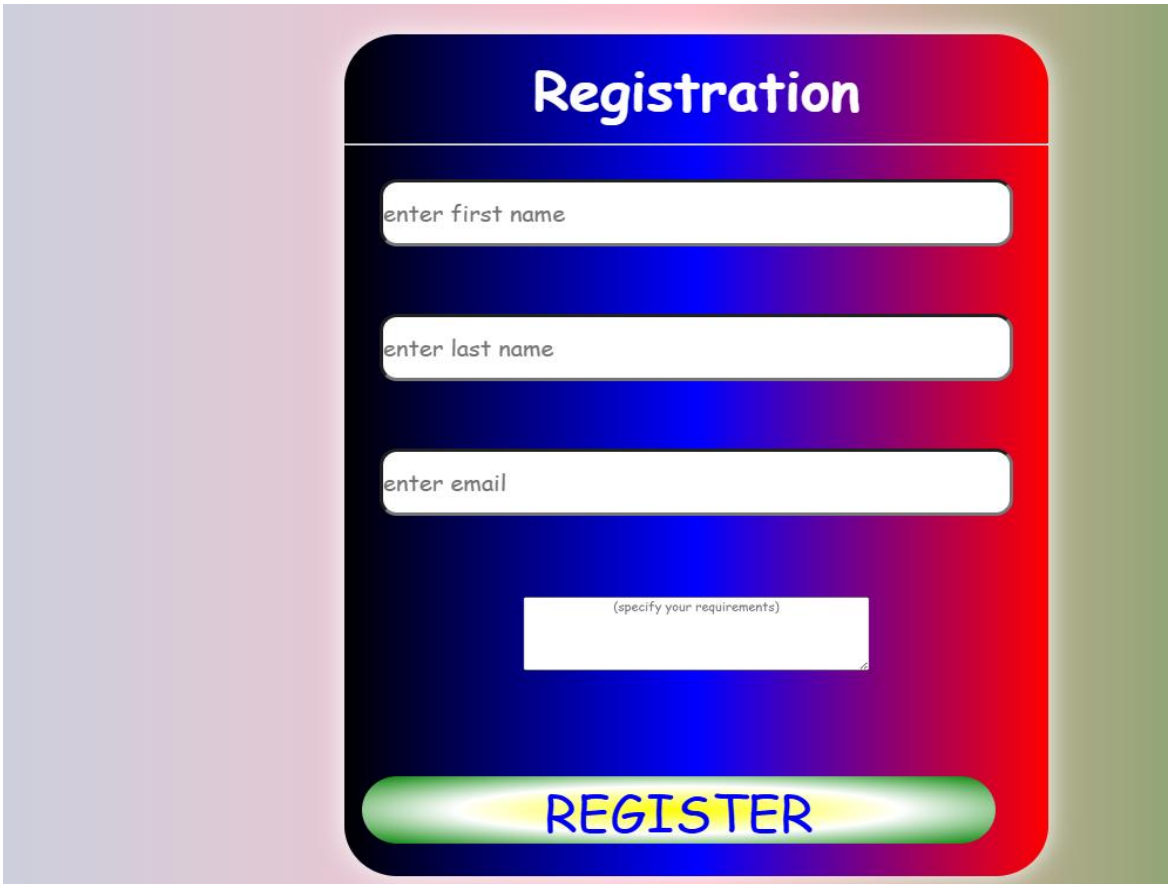


Fig 3.26 Animation effects applied using Animate.css

6. Challenges Faced

- Designing proper layout structure using HTML tags
- Aligning content sections properly using CSS
- Managing navigation links between pages
- Maintaining consistent UI design across sections
- Organizing content into structured blocks

7. Tools & Environment Used

- Programming Language: HTML, CSS
- IDE: Visual Studio Code
- Browser: Google Chrome / Edge
- Platform: Front-End Web Development

8. Conclusion

This project helped me understand the fundamentals of front-end web development. It improved my skills in HTML structure creation and CSS styling techniques.

The project also enhanced my ability to design structured web pages with navigation menus and content sections. Overall, it provides a strong foundation for building more advanced web applications in the future.

3.4.2 GQT Tours and Travels using HTML, CSS and Bootstrap

1. Introduction

This project is a front-end web development application developed using HTML, CSS, and JavaScript. It is designed to demonstrate interactive web page development with multiple user interface components and dynamic behaviour.

The project focuses on creating a structured and responsive web page that includes interactive elements, styling effects, and JavaScript-based functionality. It simulates a real-world web application interface and helps in understanding how frontend technologies work together to build modern websites.

2. Objectives

- To develop a responsive front-end web application
- To implement interactive user interface elements using JavaScript
- To understand DOM manipulation and event handling
- To practice real-time frontend development concepts
- To improve UI design and layout structuring skills
- To gain hands-on experience in web development

3. Features

- Responsive web design layout
- Interactive buttons and user actions
- Dynamic content updates using JavaScript
- Structured page layout using HTML elements
- Styled components using CSS
- Event-driven programming implementation

- User-friendly interface design

4. Implementation Details

4.1 Technologies Used

- HTML (HyperText Markup Language)
- CSS (Cascading Style Sheets)
- JavaScript
- Bootstrap – Used for responsive layout design and pre-built UI components such as navbar, carousel, and grid system
- Font Awesome – Used to add icons for better UI design
- Animate.css – Used to apply animation effects like fade, slide, and bounce

4.2 Project Structure

1. **index.html**
    - Main structure of the web application
    - Contains UI elements and layout
2. **style.css**
    - Handles styling, colors, spacing, and responsiveness
3. **script.js**
    - Implements interactivity and logic
    - Handles events and DOM manipulation

4.3 Key Concepts Used

- HTML page structure and semantic elements
- CSS styling and responsive design principles

- JavaScript event handling
- DOM manipulation
- Dynamic content update based on user interaction
- Input validation and UI behaviour control

4.4 UI Components Used

The application consists of various user interface components that enhance usability and user experience:

- **Navigation Bar:** Provides easy access to different sections like Home, Destinations, Gallery, and Contact
- **Carousel/Slider:** Displays travel images with automatic sliding and navigation controls
- **Search Bar:** Allows users to search for destinations dynamically
- **Cards Section:** Used to display destinations with images and descriptions
- **Gallery Section:** Showcases travel images in a grid layout
- **Contact Form:** Enables users to submit queries
- **Buttons:** Interactive buttons for booking, exploring, and navigation

5. Output / Screenshots

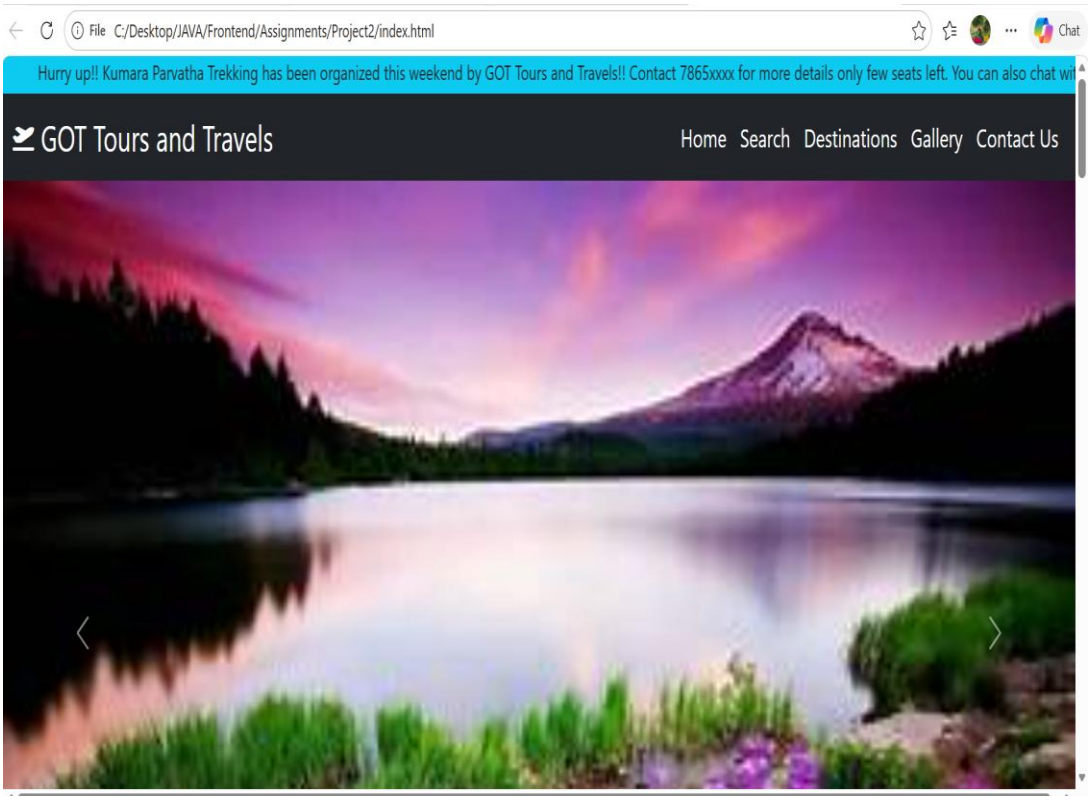


Fig 3.27 Main Web Page Layout

- **Interactive UI Components**

Fig 3.28 Navigation arrows, Carousel controls

Fig 3.29 Dynamic Content Display, Responsive View on Different Screens

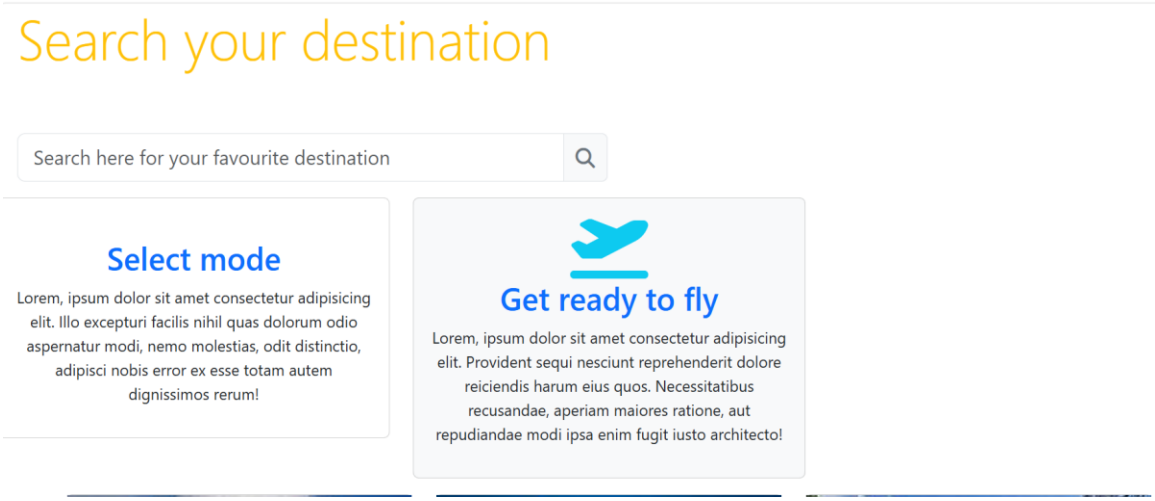


Fig 3.30 Search

Fig 3.31 Gallery

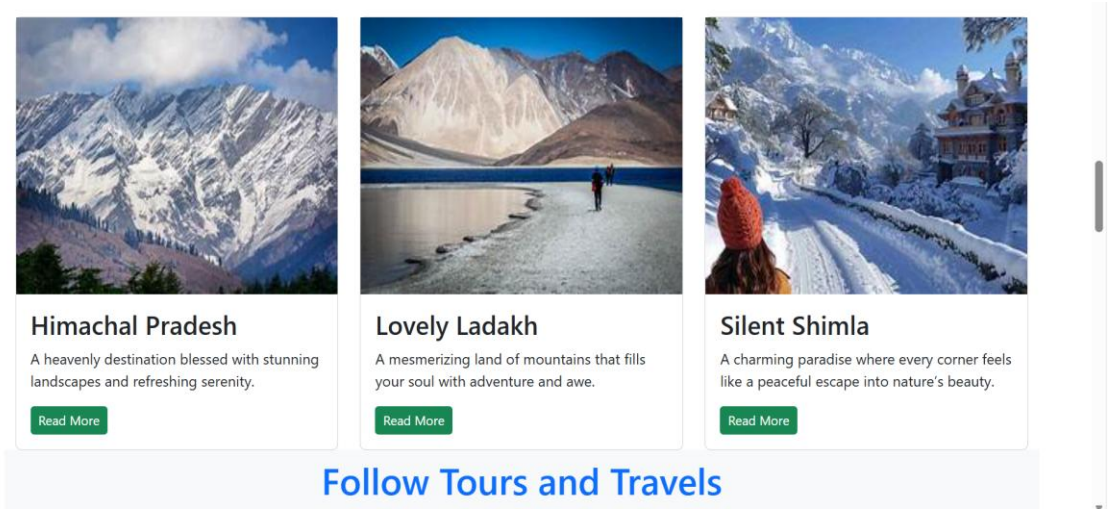


Fig 3.32 Destinations

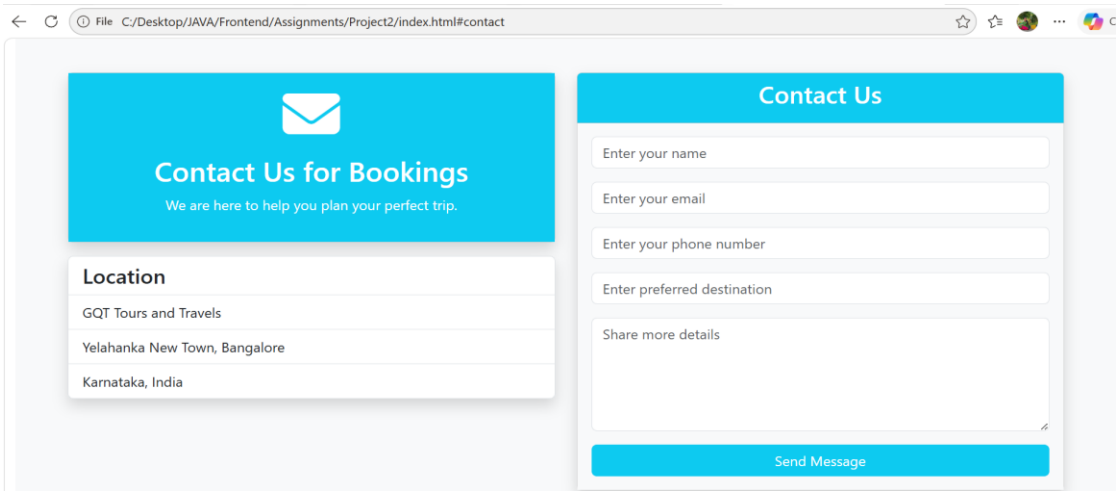


Fig 3.33 Contact Us

6. Challenges Faced

- Handling JavaScript logic for user interactions
- Designing responsive layout for multiple devices
- Maintaining clean UI structure and alignment
- Debugging DOM manipulation issues
- Ensuring proper event handling

7. Tools & Environment Used

- Programming Languages: HTML, CSS, JavaScript
- IDE: Visual Studio Code
- Browser: Google Chrome / Edge
- Platform: Front-End Web Application

8. Learning Outcomes

- Improved understanding of frontend technologies
- Gained knowledge in responsive web design
- Learned JavaScript event handling and DOM manipulation
- Enhanced UI/UX design skills
- Developed ability to build real-time web applications

9. Conclusion

This project helped me strengthen my understanding of front-end development concepts. It improved my skills in HTML structure design, CSS styling, and JavaScript functionality.

The project also enhanced my ability to build interactive and responsive web applications. Overall, it provides a strong foundation for developing advanced frontend and full stack applications in the future.

4. WEEKWISE ACTIVITY LOG

Chapter 4: ACTIVITY LOG

WEEK WISE ACTIVITY LOG

Weeks	Description of the weekly activity	Learning Outcome	Person In-charge Signature
Week-1	Introduction to internship program, overview of syllabus, installation and setup of IDE (Eclipse), basics of Core Java including variables, data types, operators, and syntax	Gained understanding of Java basics, development environment setup, and program structure	
Week-2	Studied control statements such as if-else, switch, loops (for, while, do-while), and practiced multiple basic programs	Improved logical thinking, problem-solving ability, and coding practice	
Week-3	Learned Object-Oriented Programming concepts including inheritance, polymorphism, encapsulation with examples and programs	Developed strong foundation in OOP concepts and code reusability	

Week-4	Covered advanced Core Java topics like abstraction, exception handling, and performed practice exercises and debugging	Learned error handling techniques and improved coding efficiency	
Week-5	Introduction to HTML, learned basic tags, forms, tables, and created structured web pages	Understood webpage structure and content organization	
Week-6	Worked on CSS including colors, fonts, spacing, box model, and layout design techniques	Improved UI design skills and styling of web pages	
Week-7	Developed a mini project using HTML and CSS with multiple sections and navigation	Gained hands-on experience in building static websites	

Week-8	Introduction to Bootstrap framework, grid system, components, and responsive design concepts	Learned to create mobile-friendly and responsive layouts	
Week-9	Learned JavaScript basics including variables, functions, DOM manipulation, and event handling	Understood how to add interactivity and dynamic behavior to web pages	
Week-10	Developed a mini project using JavaScript and Bootstrap with interactive features	Improved skills in dynamic UI development and user interaction handling	

Week-11	Studied Advanced Java topics such as Serialization, JDBC, and Servlets with database connectivity	Learned backend processing, data storage, and server-side programming	
Week-12	Introduction to Hibernate Framework, ORM concepts, configuration, mapping, and performing CRUD operations	Learned database interaction using ORM and simplified data persistence techniques	
Week-13	Developed a mini project using Hibernate with database connectivity and CRUD operations	Gained hands-on experience in using Hibernate for real-time application development	

Week-14	Introduction to Spring Framework, Spring Core, Dependency Injection, and basic modules	Understood Spring architecture and inversion of control concepts	
Week-15	Worked on Spring Boot, creating REST APIs, integrating with database, and application configuration	Learned to build scalable backend applications using Spring Boot	
Week-16	Final project development using Spring Boot and Hibernate integration, testing, documentation, and presentation	Gained full-stack development experience and confidence in building real-time applications.	

5. CONCLUSION

Chapter 5: CONCLUSION

1. Overall Internship Experience and Learning Journey

The internship at Global Quest Technologies provided me with valuable practical exposure in Java Full Stack Development. It was a highly enriching experience that helped me bridge the gap between theoretical knowledge and real-world application. Throughout the internship, I was introduced to various technologies including Core Java, Advanced Java, MySQL, and frontend technologies such as HTML, CSS, Bootstrap, and JavaScript.

The learning journey was well-structured, starting from basic programming concepts and gradually progressing to more advanced topics. By actively participating in daily coding challenges, weekly assignments, and guided sessions, I was able to strengthen my understanding of programming fundamentals and software development practices. This step-by-step approach made the learning process effective and easy to follow.

The internship environment was supportive and encouraging, which helped me stay motivated and consistent. Regular practice and mentor guidance played an important role in improving my confidence and technical abilities. Overall, this internship provided a strong foundation for my career in software development.

2. Technical Skills and Project Implementation

During the internship, I gained hands-on experience in multiple technologies and significantly improved my technical skills. I developed a strong understanding of Core Java concepts such as variables, control structures, object-oriented programming, and exception handling. In addition, I worked with Advanced Java technologies including JDBC, Servlets, and Hibernate, which helped me understand backend development and database connectivity. On the frontend side, I learned HTML, CSS, Bootstrap, and JavaScript, which enabled me to design and develop responsive and interactive web pages. I worked on mini projects and real-time applications that involved both frontend and backend integration. These projects helped me understand how to build structured and modular applications. By working on these projects, I improved my coding efficiency, logical thinking, and problem-solving abilities. I also gained experience in debugging errors, optimizing code, and implementing user-friendly features. The exposure to real-time application development helped me understand the complete workflow of full stack development.

3. Professional Development and Future Scope

Apart from technical skills, this internship also contributed to my overall professional development. I improved my communication skills through discussions and project presentations. Working on assignments and projects helped me develop time management skills and the ability to meet deadlines effectively.

The internship also enhanced my problem-solving approach, as I learned to analyze requirements and implement appropriate solutions. It gave me confidence in handling real-world development tasks and working independently as well as in a team environment.

Looking ahead, I plan to further enhance my knowledge by learning advanced frameworks such as Spring and Spring Boot, and by gaining deeper understanding of full stack development and system design. I aim to build more real-time applications to strengthen my practical experience.

In conclusion, this internship was a valuable learning experience that improved both my technical and professional skills. It has prepared me to face future challenges and pursue a successful career in the software industry.

REFERENCES

1. Official Website – https://gqtech.in/
2. JavaTPoint, *"Java Tutorial."*
   Available: https://www.tpointtech.com/java-tutorial
3. GeeksforGeeks, *"Java Programming Language."*
   Available: https://www.geeksforgeeks.org/java/
4. GeeksforGeeks, *"Java Swing Tutorial."*
   Available: https://www.geeksforgeeks.org/java-swing/
5. Oracle Corporation, *"Java Platform, Standard Edition Documentation (JDK 8)."*
   Available: https://docs.oracle.com/javase/8/docs/
6. W3Schools, *"Java Tutorial."*
   Available: https://www.w3schools.com/java/
7. TutorialsPoint, *"Java Swing Tutorial."*
   Available: https://www.tutorialspoint.com/swing/index.htm
8. Global Quest Technologies, *"Training Materials and Internship Resources."*